

FUXA 完整文档

FUXA 是一个强大的基于 Web 的软件，可用于快速构建和部署可扩展的 SCADA、HMI、Dashboard 或 IIoT 系统。通过 FUXA，您可以为您的机器创建具有个性化设计的现代流程可视化，实时显示数据，以及控制自动化工业工厂的仪器。

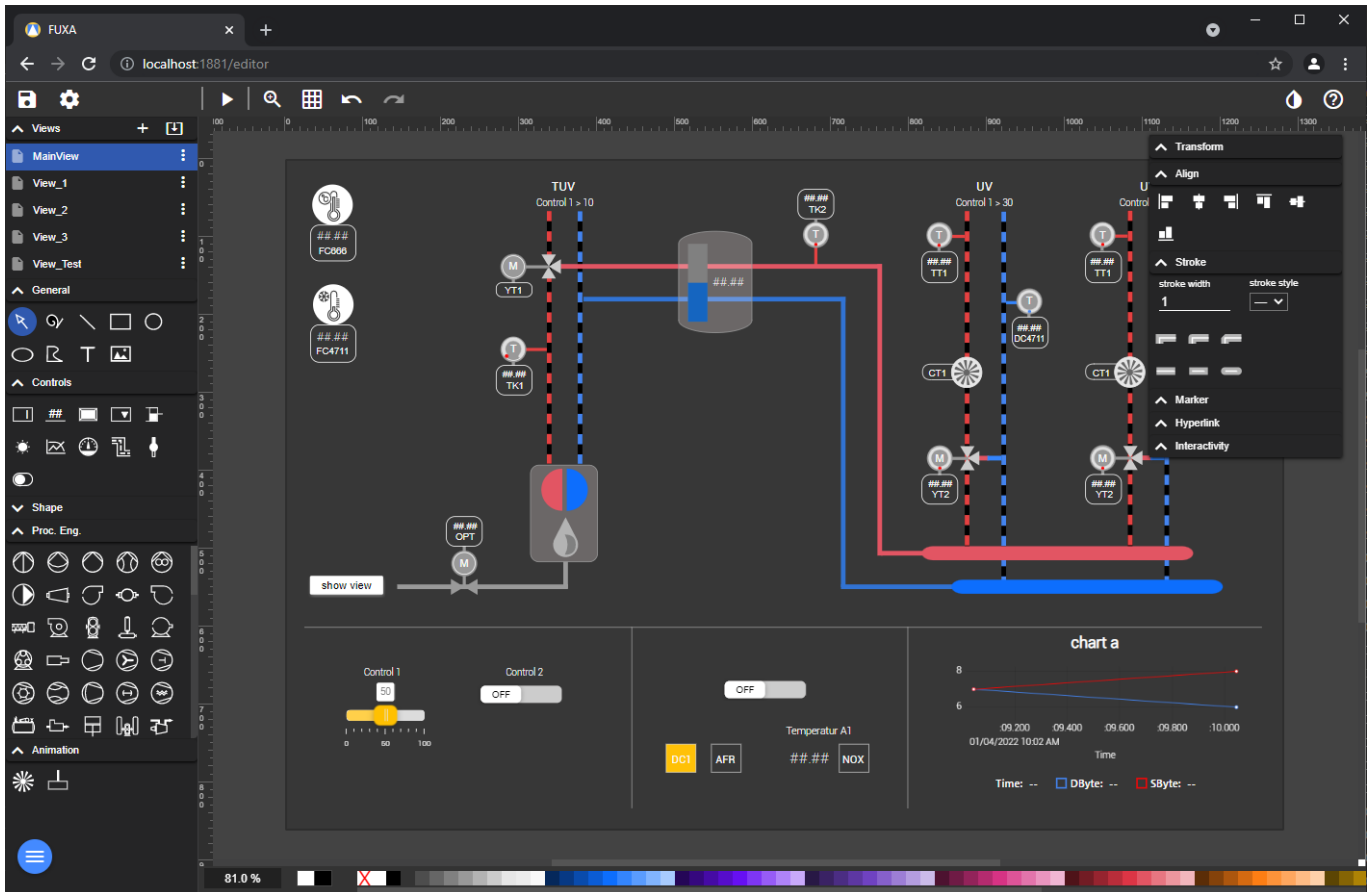
目录

1. [首页](#)
2. [入门指南](#)
3. [安装与运行](#)
4. [设备和标签](#)
5. [绑定控件到视图](#)
6. [绑定图形到视图](#)
7. [创建视图](#)
8. [图表控件](#)
9. [UI 布局](#)
10. [报警设置](#)
11. [自定义图形](#)
12. [管道动画](#)
13. [调度器](#)
14. [Node-RED 集成](#)
15. [项目保存/加载](#)
16. [事件配置](#)
17. [复用视图](#)
18. [脚本配置](#)
19. [ODBC 配置](#)
20. [小部件](#)
21. [WebSocket](#)
22. [设置](#)
23. [技巧和窍门](#)

1. 首页

欢迎使用 FUXA Wiki!

FUXA 是一个强大的**基于 Web** 的软件，可用于快速构建和部署可扩展的 SCADA、HMI、Dashboard 或 IIoT 系统。通过 FUXA，您可以为您的机器创建具有个性化设计的现代流程可视化，实时显示数据，以及控制自动化工业工厂的仪器。

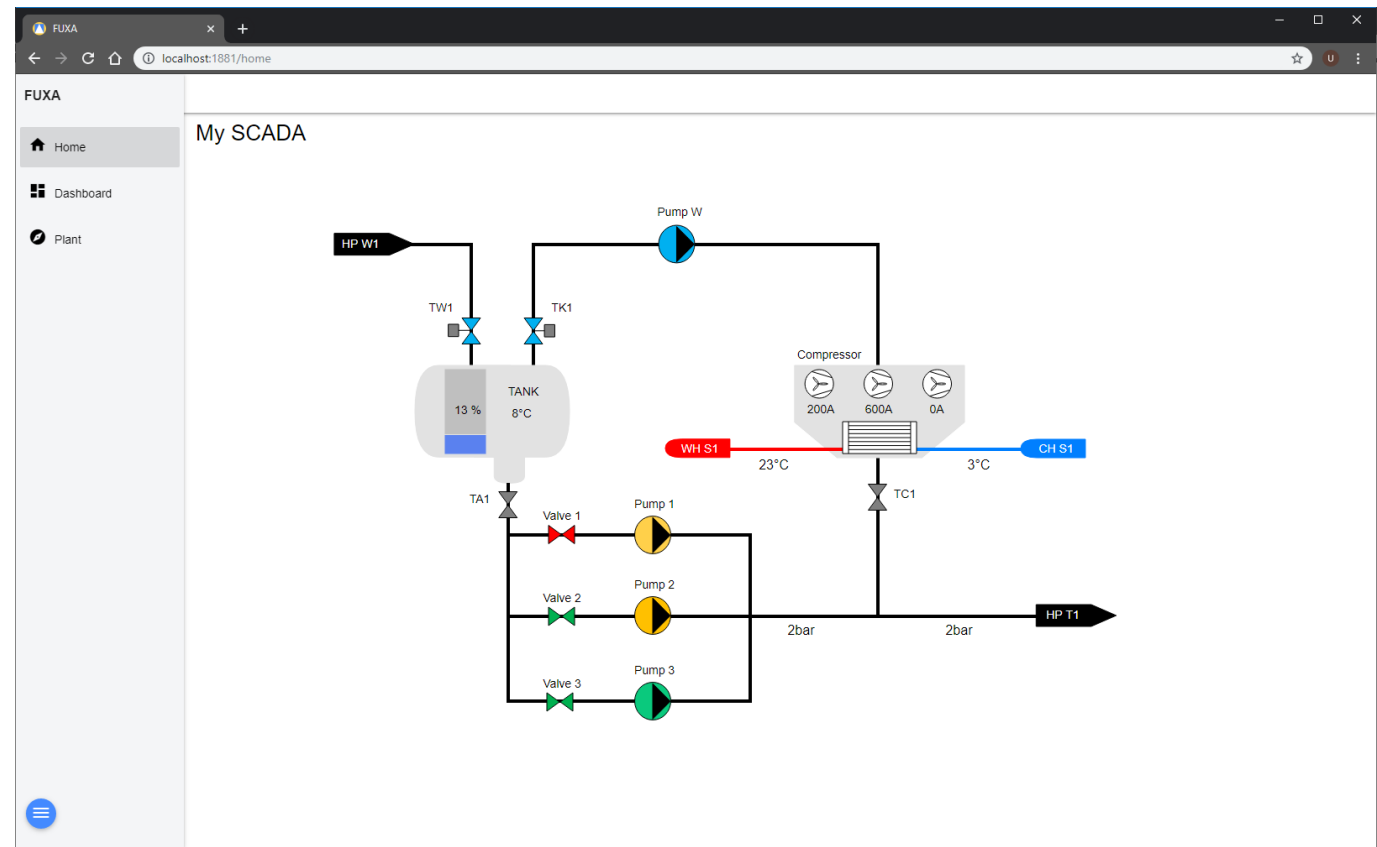


FUXA 支持常用的通信标准，如西门子 S7 协议和 OPC UA，可连接第三方 OPC 服务器。支持的通信标准列表可以通过开发额外的驱动程序来扩展。

FUXA 的软件模型基于 Node.js 运行时，您可以通过 Web 浏览器访问编辑器来创建您的应用（SCADA/HMI/Dashboard），并作为客户端运行您的可视化。

用户界面

FUXA 由两个不同的视图组成：用于编辑项目的 FUXA 编辑器和用于显示可视化项目结果的 FUXA 视图。



功能特点

FUXA 不需要任何运行时许可证。您可以构建任意数量和规模的 HMI 项目，无需担心运行时的许可问题。

通信协议

FUXA 平台包含连接器，可实现与西门子 PLC 和 OPC UA 数据源的直接通信：

- OPC UA 连接客户端
- S7 协议，通过以太网与西门子 CPU 200、300、400、1200 和 1500 通信
- Modbus RTU/TCP、BACnet IP、MQTT、Ethernet/IP（Allen Bradley）、WebAPI

跨平台全栈

该系统设计用于在各种操作系统上运行，安装简便。后端使用 Node.js 开发。用户界面是可扩展的 HTML5 Web 前端，使用 Web 技术（HTML5、CSS、JavaScript、Angular、SVG）开发，兼容所有现代浏览器。

2. 入门指南

本教程旨在演示 FUXA 最流行功能的基本用法。您将学习如何：

- 将设备连接到 FUXA
- 将数据从设备推送到 FUXA
- 构建实时 GUI SCADA/HMI/Dashboard
- 定义阈值并触发报警

前置条件

您需要让 FUXA 服务器运行起来。按照[安装指南](#)安装 FUXA。FUXA UI 可通过以下 URL 访问：
<http://localhost:1881>。

FUXA Web 服务器主要提供两个页面：

- 面向最终用户的可视化页面 <http://localhost:1881/home>
- 用于项目和设计的编辑器 <http://localhost:1881/editor>

步骤 1. 连接设备并配置标签

要将设备连接到 FUXA，您需要先获取设备凭据。FUXA 支持各种设备：Modbus RTU/TCP、Siemens S7 Protocol、OPC-UA、BACnet IP、MQTT、Ethernet/IP（Allen Bradley）。连接到设备后，您可以配置实时值的订阅、标签、传感器等。请参阅配置指南[设备和标签](#)

步骤 2. 创建可视化

我们将创建一个视图并添加最流行的部件。请参阅以下说明。

步骤 2.1 创建空视图

创建视图

步骤 2.2 绑定控件到视图

绑定控件

步骤 2.3 绑定图形到视图

绑定图形

步骤 2.4 绑定图表控件到视图

图表控件

步骤 3. 配置 UI 布局

UI 布局

步骤 4. 配置报警

设置报警

步骤 5. 激活并创建客户用户

提示和技巧

3. 安装与运行

FUXA 使用 NodeJS（后端）和 Angular（前端）开发。

预构建的 Electron 应用（第一种方式）

您需要登录 GitHub 才能访问 Electron Action Builds 的下载按钮，点击 workflow 并向下滚动到 Artifacts，点击适合您系统的下载图标。

Electron Action Builds

Artifacts			
Produced during runtime			
Name	Size	Digest	
 FUXA-macos-latest-arm64	138 MB	sha256:698a44f53d97bd2dc3a43eedf9c1f29998b31624a5ce82f78cfd69947953...	  
 FUXA-macos-latest-x64	144 MB	sha256:0058bec8e5cfe1f8896b59dd2f2c7f7485fb95a6d95b6b6532d7fd2bc431...	  
 FUXA-ubuntu-latest-arm64	147 MB	sha256:8381d49f9ed7dcecc7b24359af4d13cb55ddead286dad33f41d379e77ef262...	  
 FUXA-ubuntu-latest-x64	147 MB	sha256:db6dce995593c3835c3a7bd9d7b3c8b44df838aaff8d15a2565e73b4ee6a9a...	  
 FUXA-windows-latest-x64	115 MB	sha256:4336e81fe1ccb4a9e98dbf7e7a9114881257e7bc8216d8e26857a2bfe8d9...	  

安装非 Electron 版本后使用 Web 浏览器访问

完成任一种安装过程后，您可以通过默认端口 1881 和 Web 服务器 IP 地址（localhost 或主机 IP）访问 Fuxa UI，首先尝试 localhost:1881 或主机 IP:1881。

Docker Compose（第二种方式）

安装 Docker

```
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh ./get-docker.sh
```

```
cd
mkdir docker
mkdir fuxa
cd docker/fuxa
sudo nano docker-compose.yml
```

将以下内容输入到 compose 文件中：

```
version: '3.5'

services:
  fuxa:
    image: frangoteam/fuxa:latest
    network_mode: "host"
    volumes:
      - ./fuxa_appdata:/usr/src/app/FUXA/server/_appdata
      - ./fuxa_db:/usr/src/app/FUXA/server/_db
      - ./fuxa_logs:/usr/src/app/FUXA/server/_logs
    environment:
      - TZ=America/New_York
    restart: always
```

注意：此 Compose 文件使用主机网络模式，请参阅 [Docker Networks](#) 了解更多详情，主机模式允许 Docker 容器与主机机器位于同一网络。这对于直接访问 PLC/数据库等效果最佳，但有一些限制，仅适用于 Linux 系统，如果需要在 Windows 上运行，则需要使用桥接网络模式。

```
network_mode: "bridge"
```

启动 Docker Compose

```
sudo docker compose up -d
```

Docker（第三种方式）

```
docker pull frangoteam/fuxa:latest
docker run -d -p 1881:1881 frangoteam/fuxa:latest
```

持久化存储应用数据（项目）、daq（标签历史）、日志和资源图像

```
docker run -d -p 1881:1881 -v fuxa_appdata:/usr/src/app/FUXA/server/_appdata -v
fuxa_db:/usr/src/app/FUXA/server/_db -v fuxa_logs:/usr/src/app/FUXA/server/_logs -
v fuxa_shapes:/usr/src/app/FUXA/client/assets/lib/svgeditor/shapes -v
fuxa_images:/usr/src/app/FUXA/server/_images frangoteam/fuxa:latest
```

从源码构建自定义 Docker 镜像（第四种方式）

这将从最新的 master 分支构建（您可以编辑 docker 文件来更改分支）

```
cd
mkdir docker
mkdir fuxa
cd docker/fuxa
mkdir fuxa-build
cd fuxa-build
```

```
wget https://raw.githubusercontent.com/frangoteam/FUXA/master/Dockerfile
```

```
sudo docker build -t fuxa-custom-image-name --no-cache .
```

构建完成后，您现在可以使用自定义镜像，例如：

```
version: '3.5'

services:
  fuxa:
    image: fuxa-custom-image-name
```

```
network_mode: "host"
volumes:
  - ./fuxa_appdata:/usr/src/app/FUXA/server/_appdata
  - ./fuxa_db:/usr/src/app/FUXA/server/_db
  - ./fuxa_logs:/usr/src/app/FUXA/server/_logs
environment:
  - TZ=America/New_York
restart: always
```

使用 Node 和 NPM 安装

您需要已安装 [Node](#) Version 18。

警告：在 Linux 上使用 Node.js Version 18 安装可能具有挑战性。如果您不打算通过 S7（node-snap7 库）与西门子 PLC 通信，可以从 [NPM @frangoteam/fuxa-min](#) 安装。

NPM（第五种方式）

从 [NPM](#) 安装

```
npm install -g --unsafe-perm @frangoteam/fuxa
fuxa
```

最新版本（第五种方式）

[下载最新版本](#) 并解压

警告：在 Linux 上使用 Node.js Version 18 安装可能具有挑战性。如果您不打算通过 S7 与西门子 PLC 通信，可以从 server/package.json 中删除 node-snap7 库。

```
cd ./server
npm install
npm start
```

打开浏览器（最好使用 Chrome）并导航到 <http://localhost:1881>

设置为 HMI（仅限 Linux）

这仅在 Linux 上经过测试，但某些步骤可能在 Windows 上也能工作。

我们将使用 electron 而不是 Web 浏览器，因为它不那么臃肿，并且适用于触摸屏（可以禁用右键菜单等）。

该应用仅访问已运行的 Fuxa Web 服务器。

```
sudo apt-get update
curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash -
sudo apt-get install -y nodejs
```

通过检查 Node.js 和 npm 的本来验证安装:

```
node -v
```

```
npm -v
```

```
sudo npm install -g electron --unsafe-perm=true --allow-root
```

```
cd /opt
sudo mkdir electron
cd electron
sudo mkdir fuxa-electron
cd fuxa-electron
```

```
sudo npm init -y
```

```
sudo npm install electron --save-dev
```

```
sudo nano main.js
```

```
const { app, BrowserWindow } = require('electron');

function createWindow() {
  const win = new BrowserWindow({
    //width: 800,
    //height: 600,
    fullscreen: true, // 启用全屏模式
    webPreferences: {
      nodeIntegration: false,
      contextIsolation: true,
    }
  });
}

win.loadURL('http://localhost:1881'); // 替换为您的 Web 服务器 URL
}

app.whenReady().then(createWindow);
```

```
sudo nano package.json
```

```
{
  "name": "fuxa-electron",
  "version": "1.0.0",
  "main": "main.js",
  "scripts": {
    "start": "electron ."
  },
  "description": "",
```



```
"devDependencies": {  
  "electron": "^31.3.1"  
}
```

从目录内启动应用进行测试

```
npm start
```

使用系统服务和脚本设置自动启动

```
sudo nano /opt/fuxa-electron-startup.sh
```

```
#!/bin/bash  
cd /opt/electron/fuxa-electron  
npm start
```

```
sudo chmod +x /opt/fuxa-electron-startup.sh
```

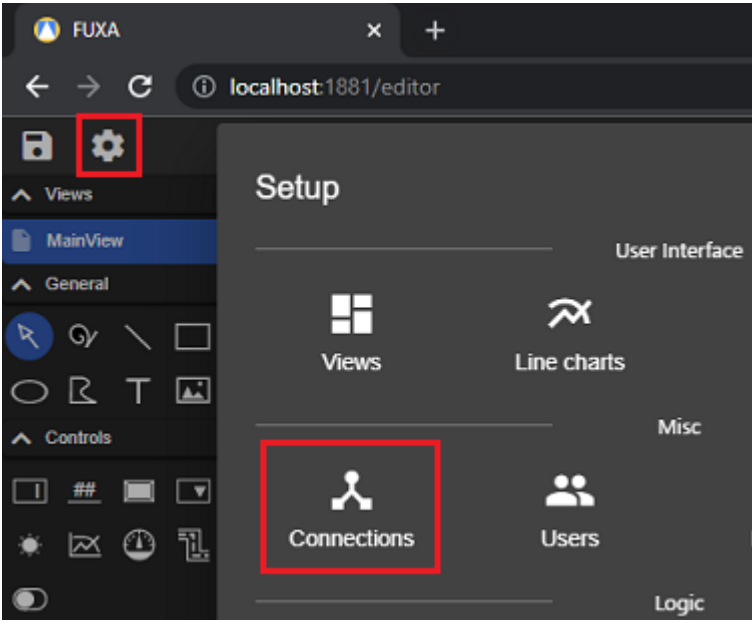
```
systemctl edit --user --force --full fuxa-electron-startup.service
```

```
[Unit]  
Description=Start Fuxa Electron Script  
After=default.target  
[Service]  
ExecStart=/opt/fuxa-electron-startup.sh  
[Install]  
WantedBy=default.target
```

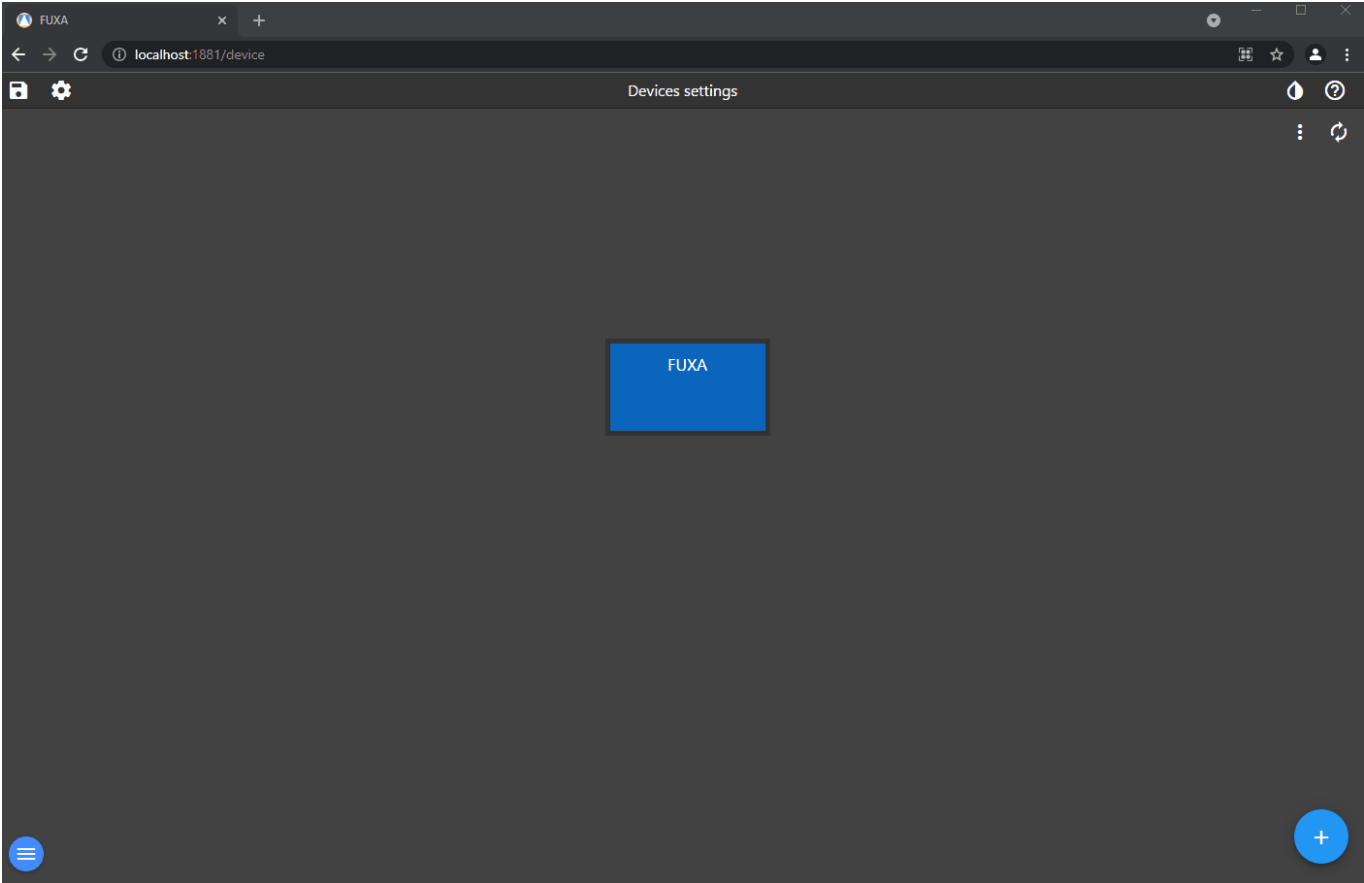
```
systemctl enable --user fuxa-electron-startup.service
```

4. 设备和标签

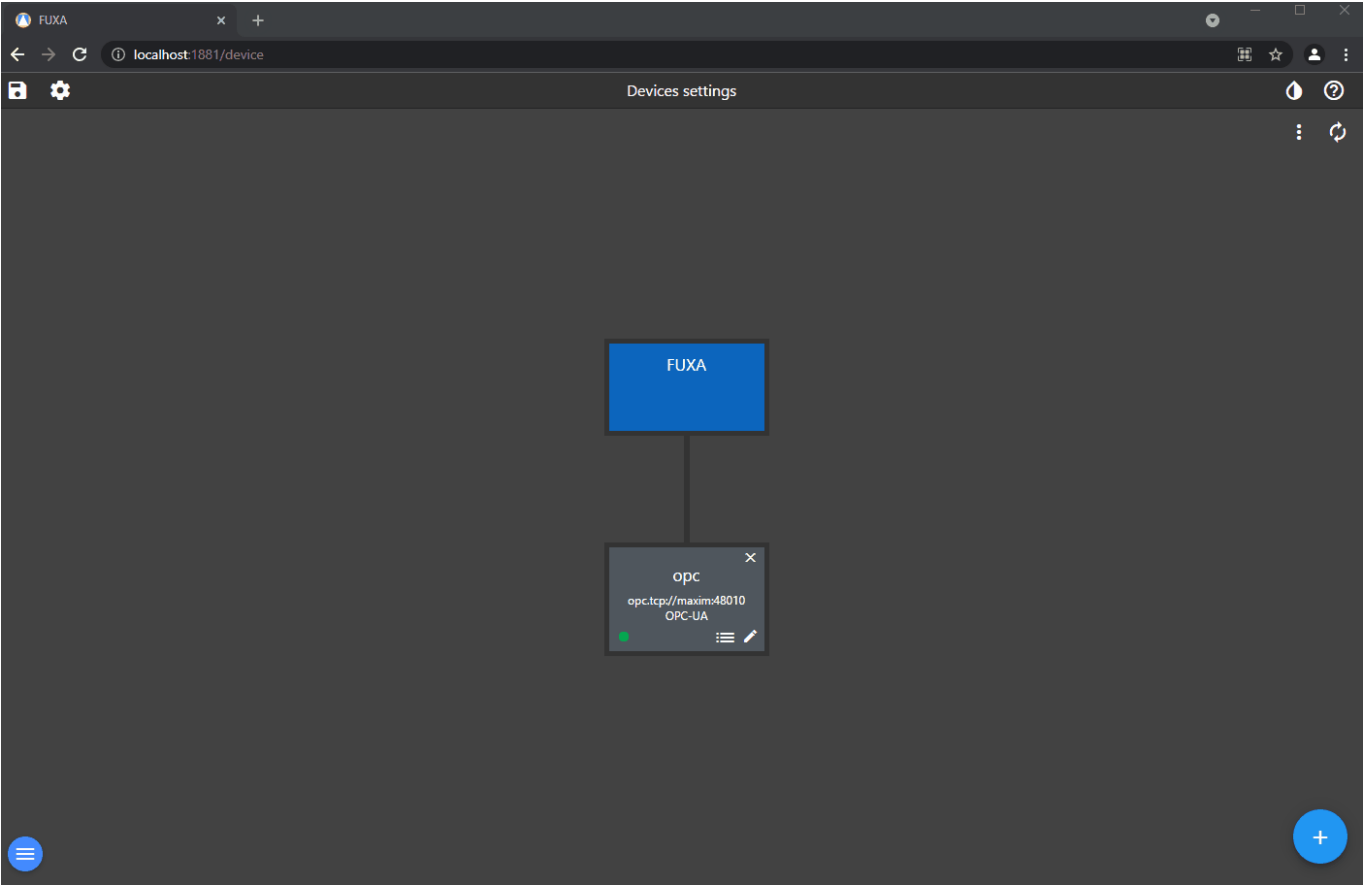
要添加您的设备和标签，请在编辑器中进入 **Connections**。



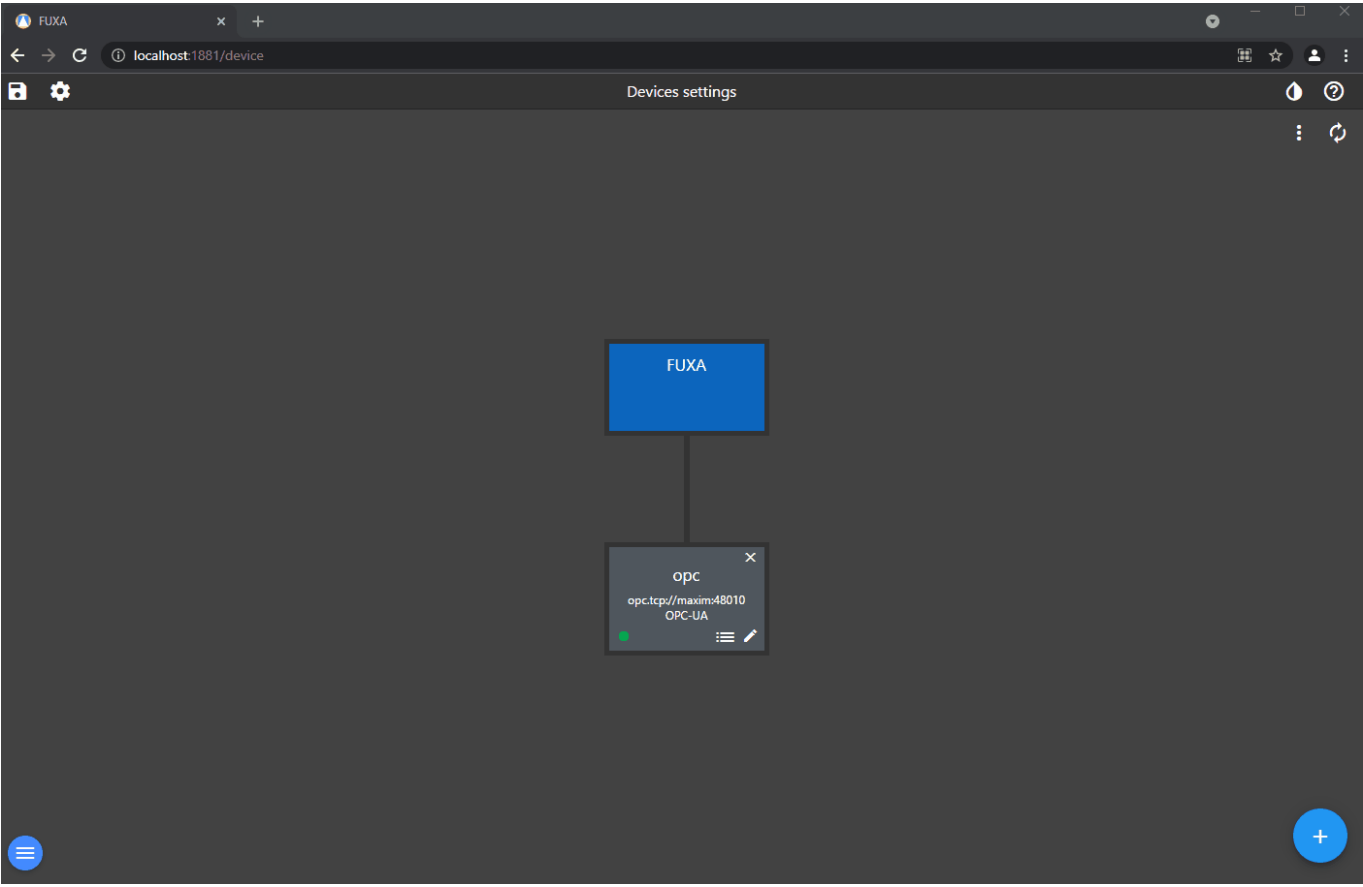
添加并连接一个 OPCUA 设备。



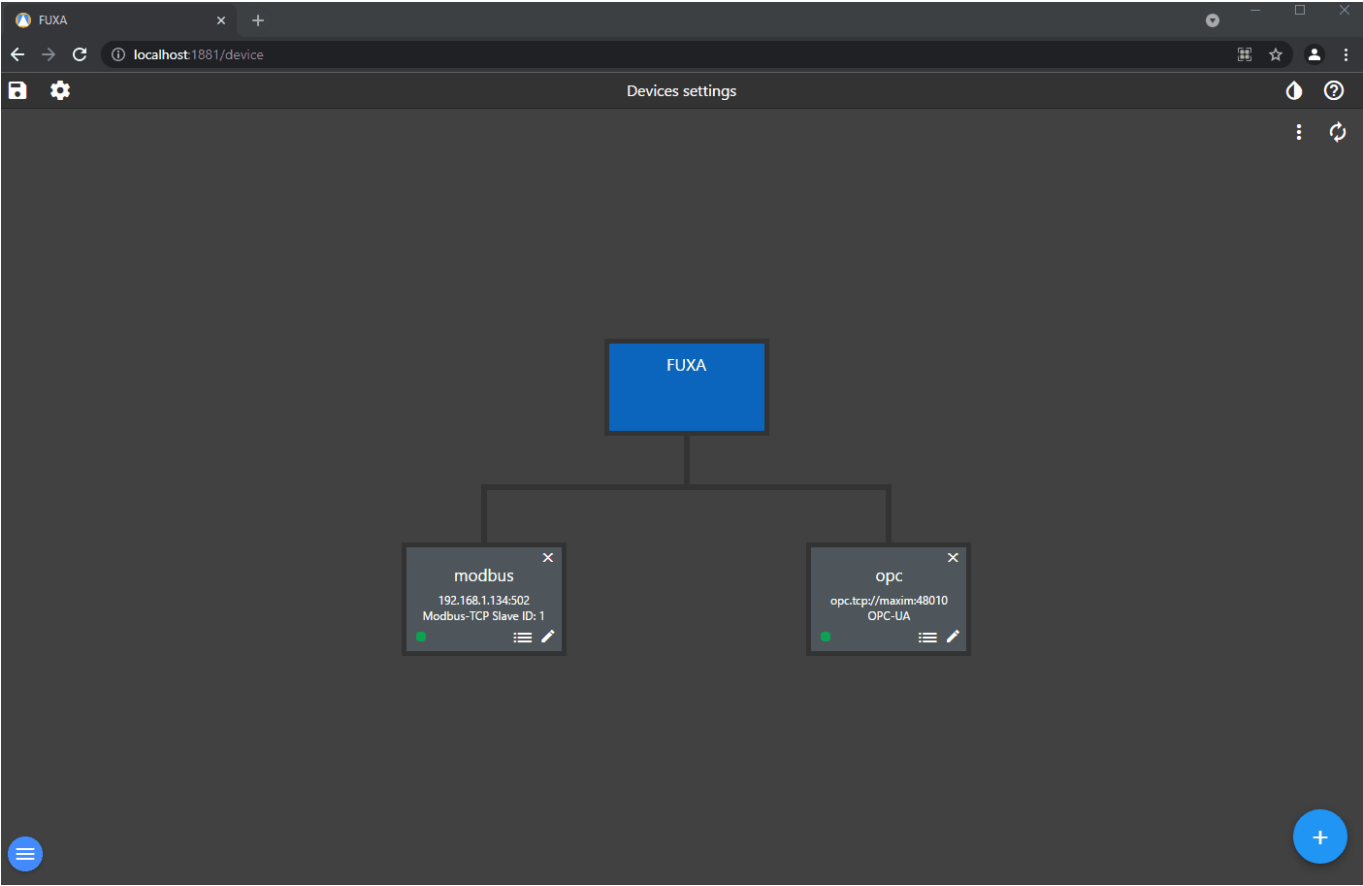
要添加 OPCUA 标签，设备必须已连接。



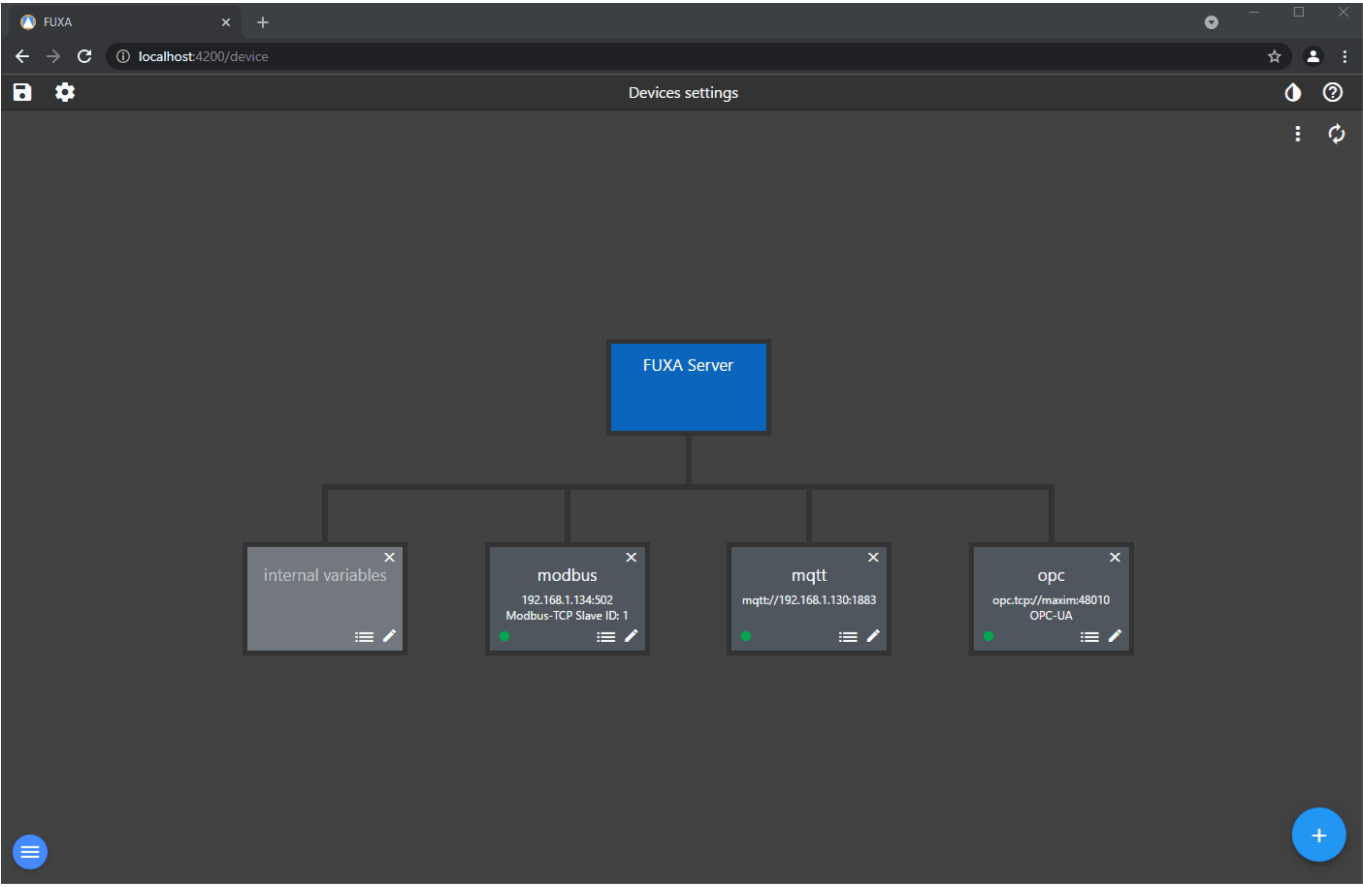
要添加 Modbus 连接，您需要在 Plugins 中安装驱动程序。



添加 **MQTT** 连接和一个主题订阅。

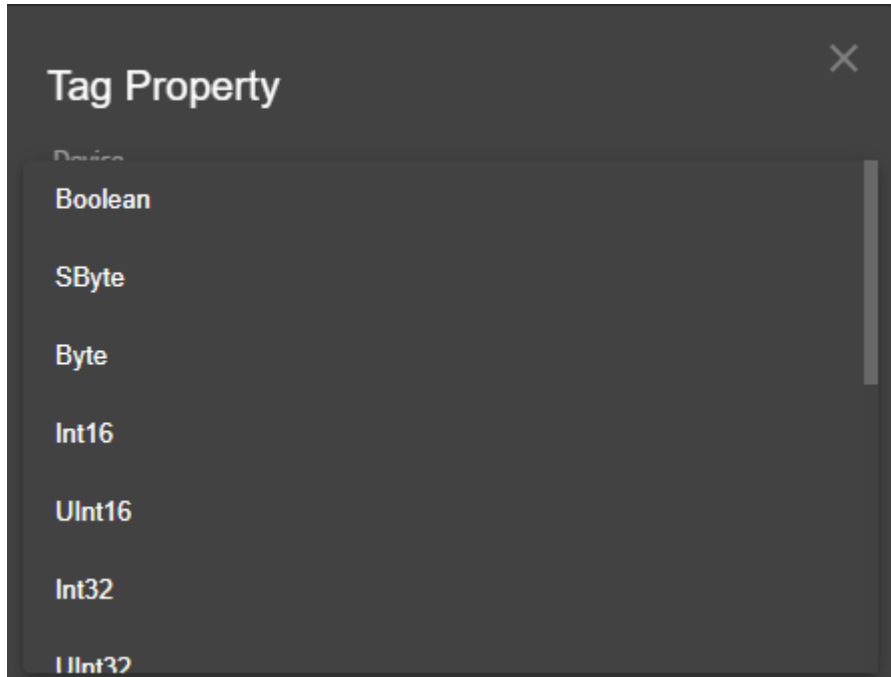


添加 **WebAPI** 连接和一个 JSON 结果的标签。



标签选项

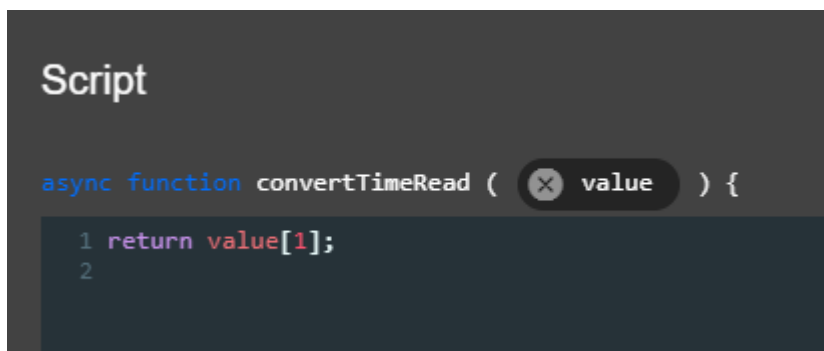
如果您有任何未定义数据类型的标签类型，可以在此设置。例如，TIME 数据类型不是已定义的类型，但基本数据类型通常是 int64 或 time 是毫秒单位的数组。



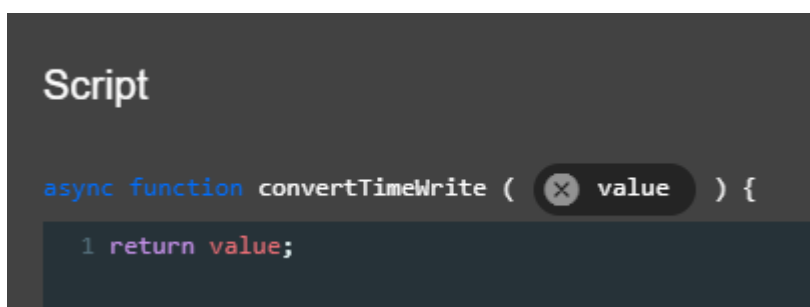
您还可以在标签选项中使用缩放脚本来执行任何数据类型转换等。例如，TIME 的数据是一个 LOW 和 HIGH 数组，我们可以创建一个简单的脚本来获取数组元素并将其作为数字类型返回。然后，如果需要，我们可以只返回实际值进行写入。

重要！ 脚本参数必须称为 value，因为有一个过滤器显示相关脚本供选择。不要在此脚本中使用注释，因为它们不受支持。

这是返回数组元素的读取脚本：

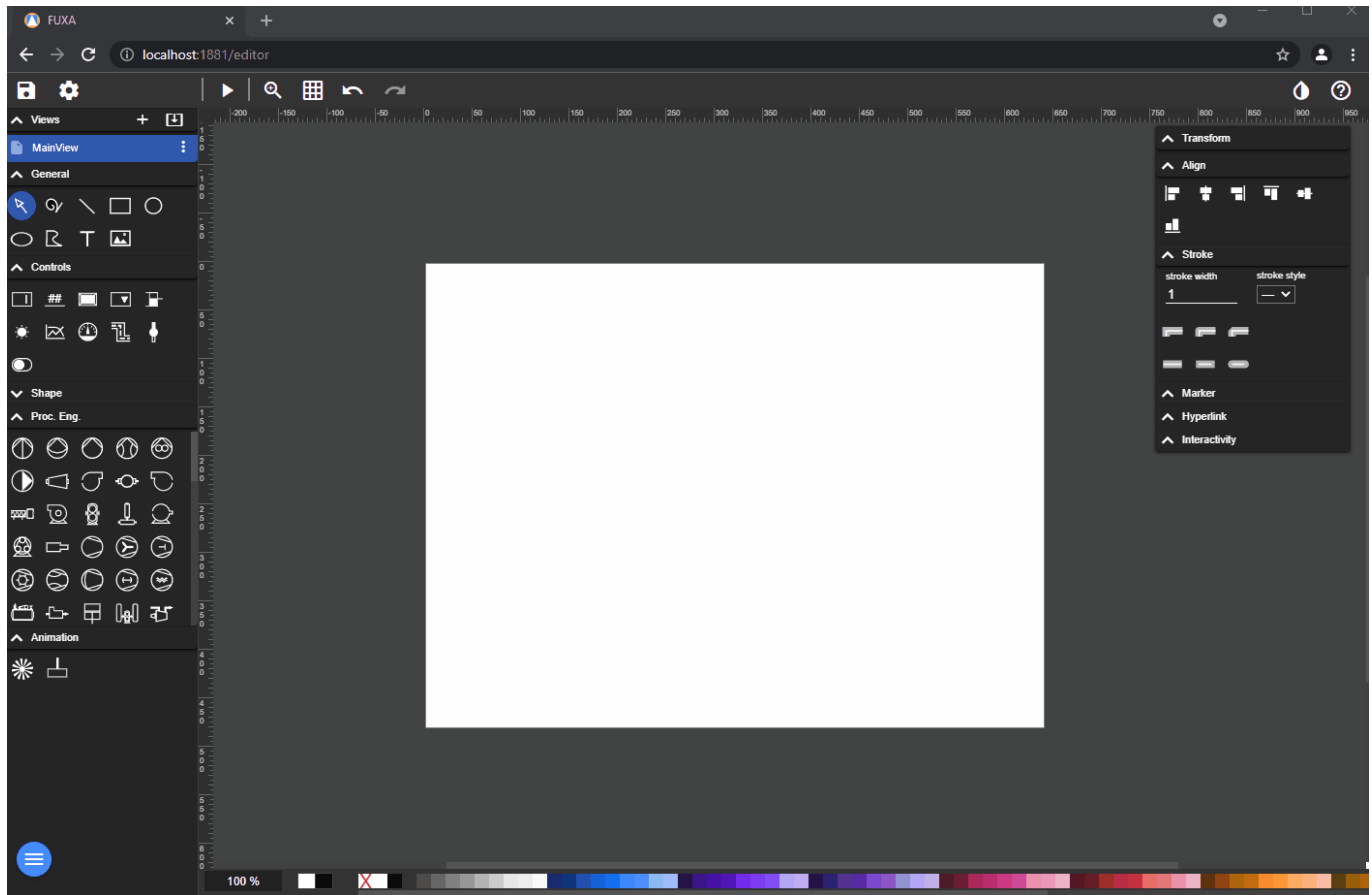


这是返回实际标签值的写入脚本：

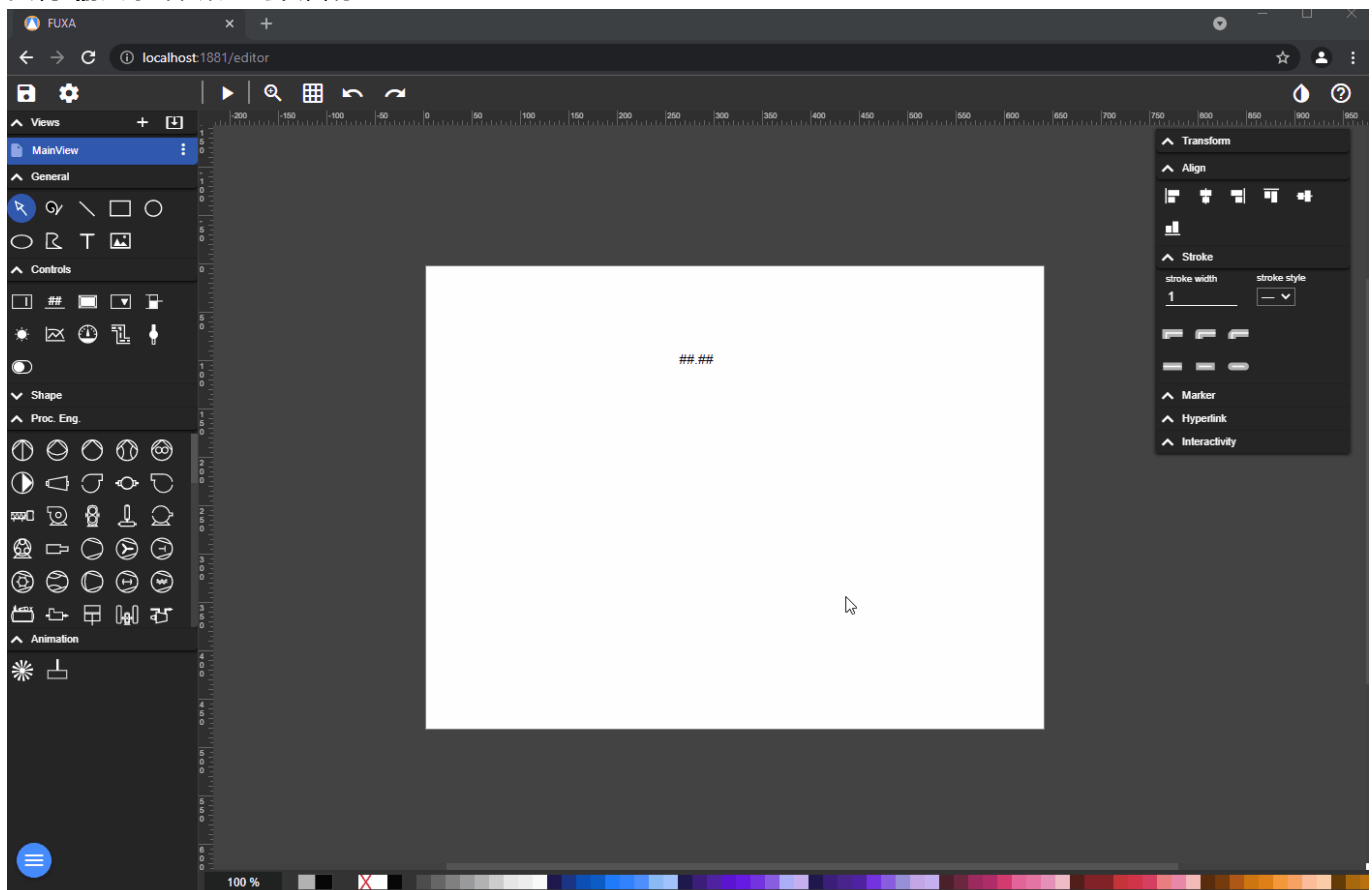


5. 绑定控件到视图

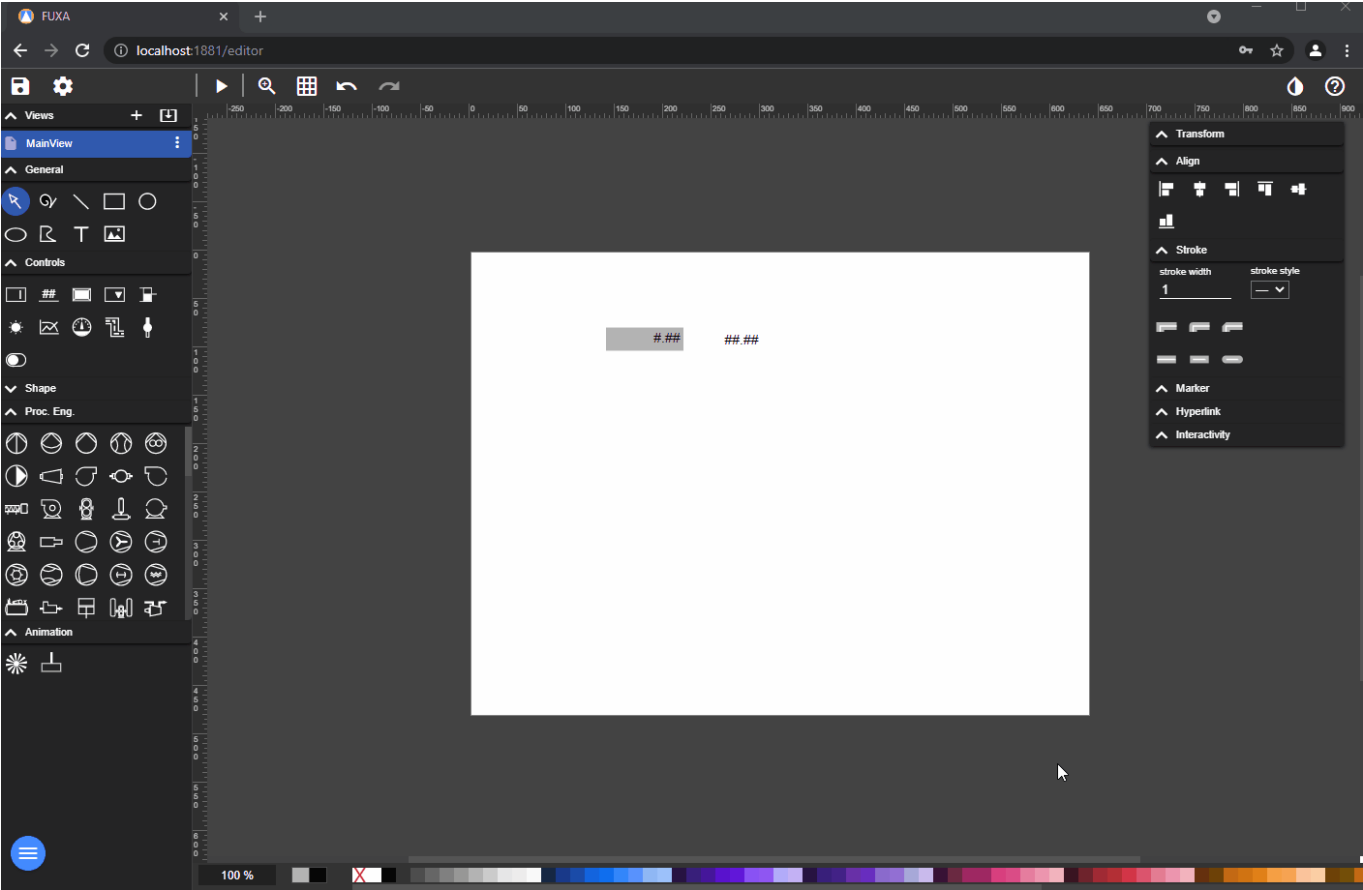
要将 **输出** 控件绑定到设备标签（变量）：在编辑器中选择视图。



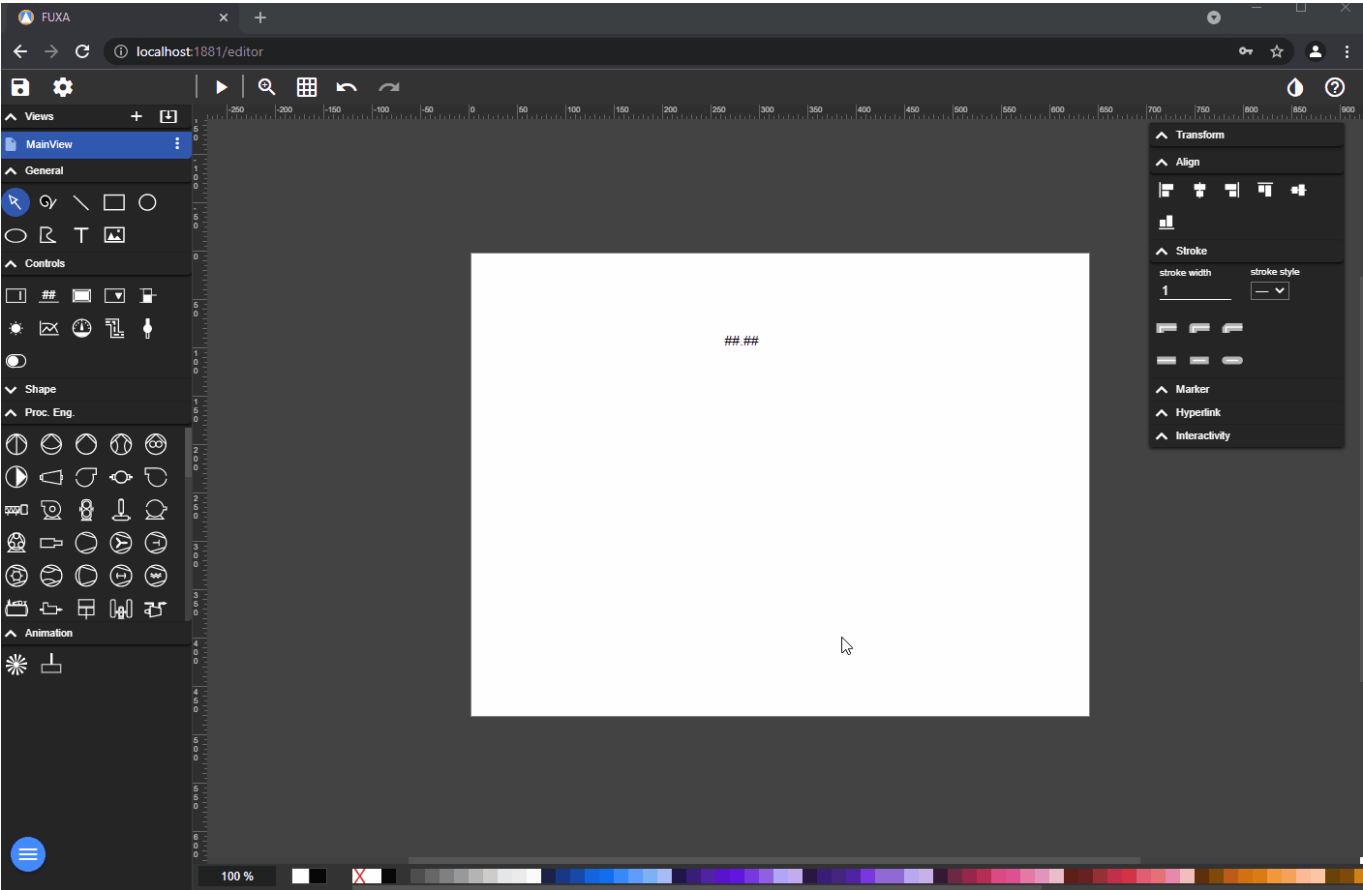
要将 **输入** 控件绑定到设备标签。



要将 **选择** 控件绑定到设备标签。

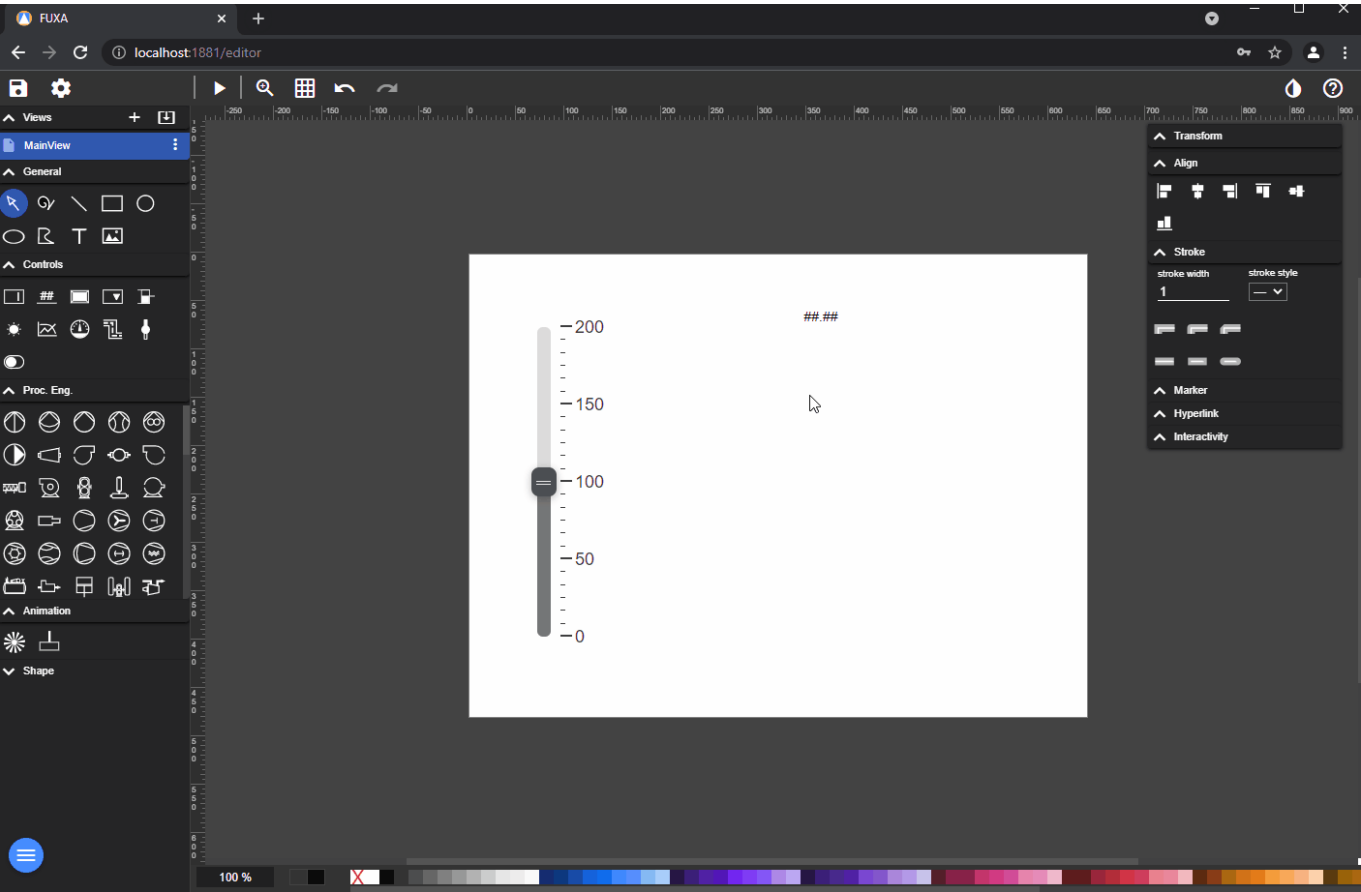


要将 **滑块** 控件绑定到设备标签。

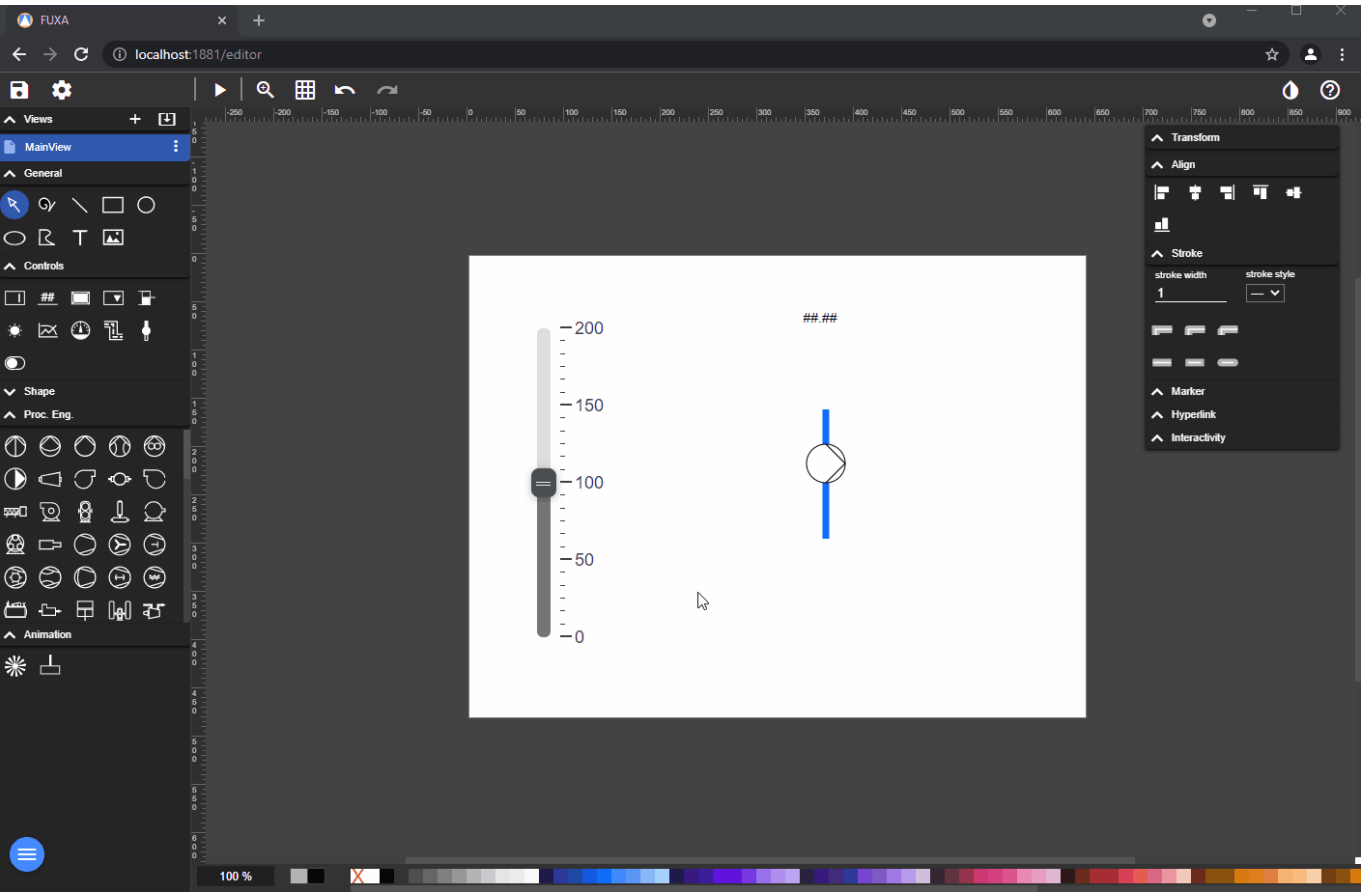


6. 绑定图形到视图

要将 **图形** 绑定到设备标签（变量）：在编辑器中选择视图。

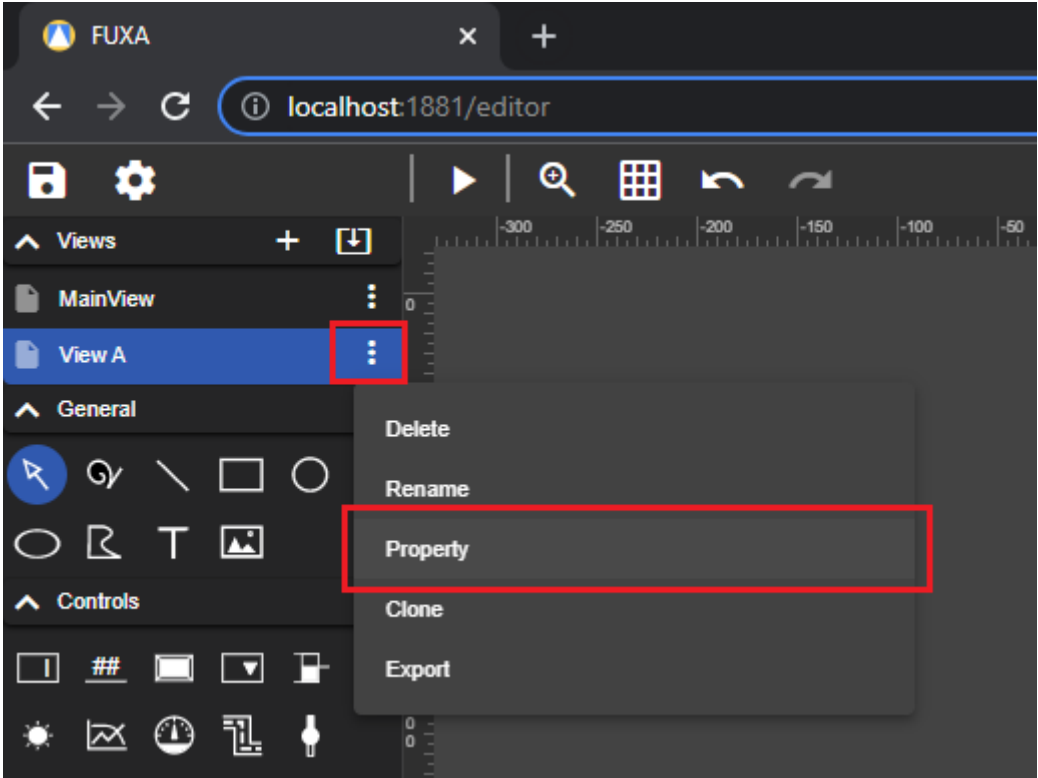
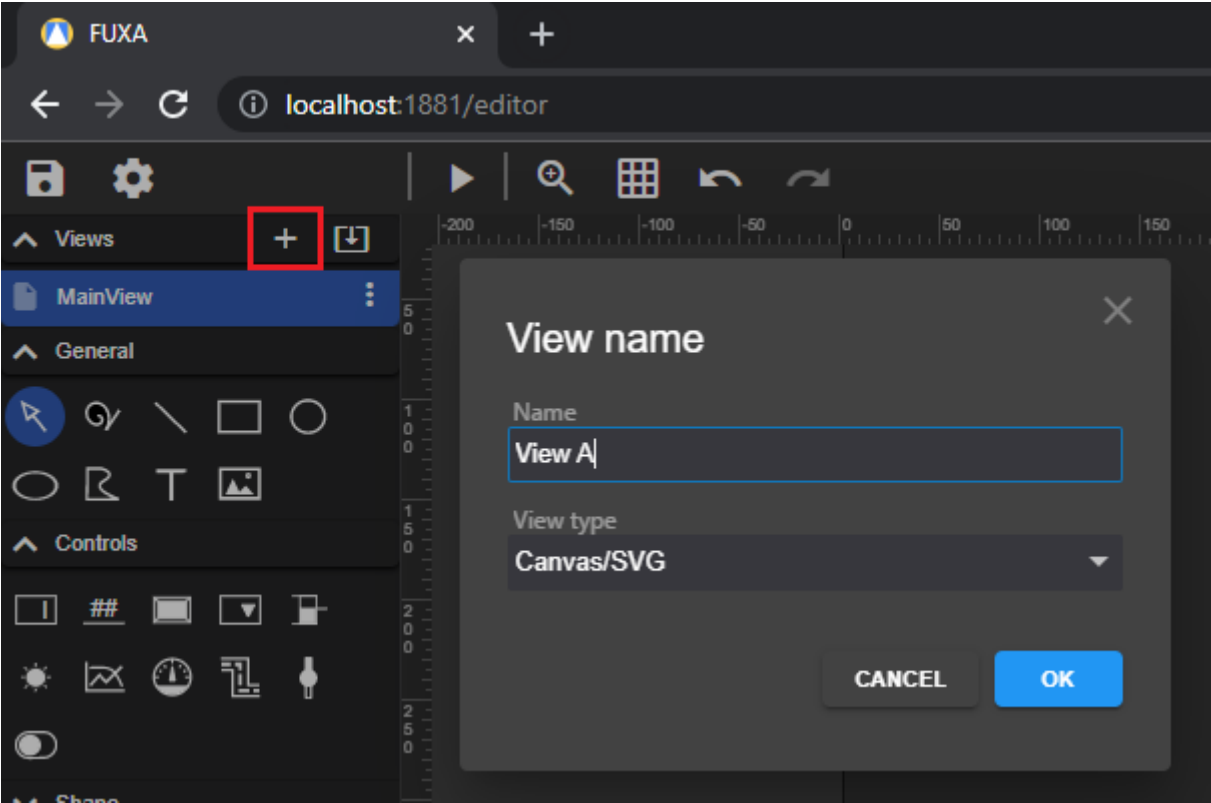


要将 **过程工程** 图形（罐体）绑定到设备标签。

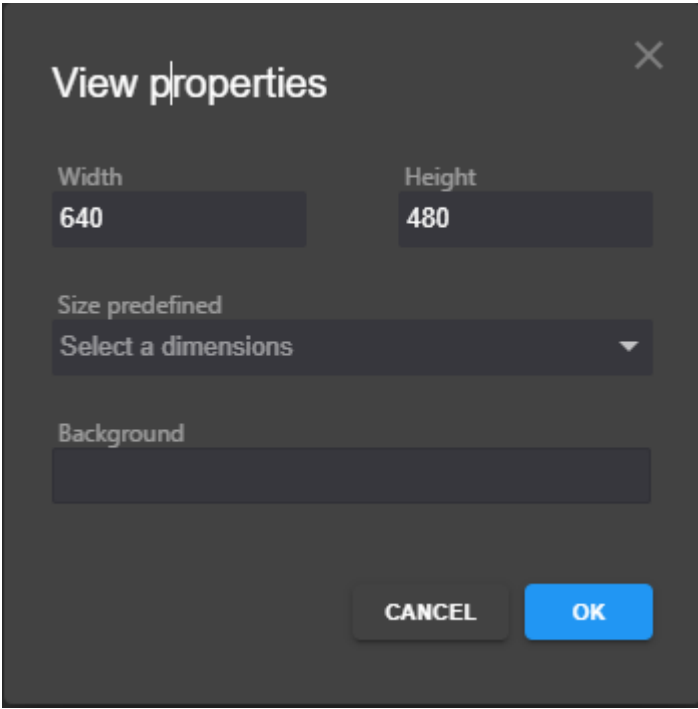


7. 创建视图

要创建视图，请在编辑器中。点击左上角的 "+" 图标。选择 "Canvas/SVG"。



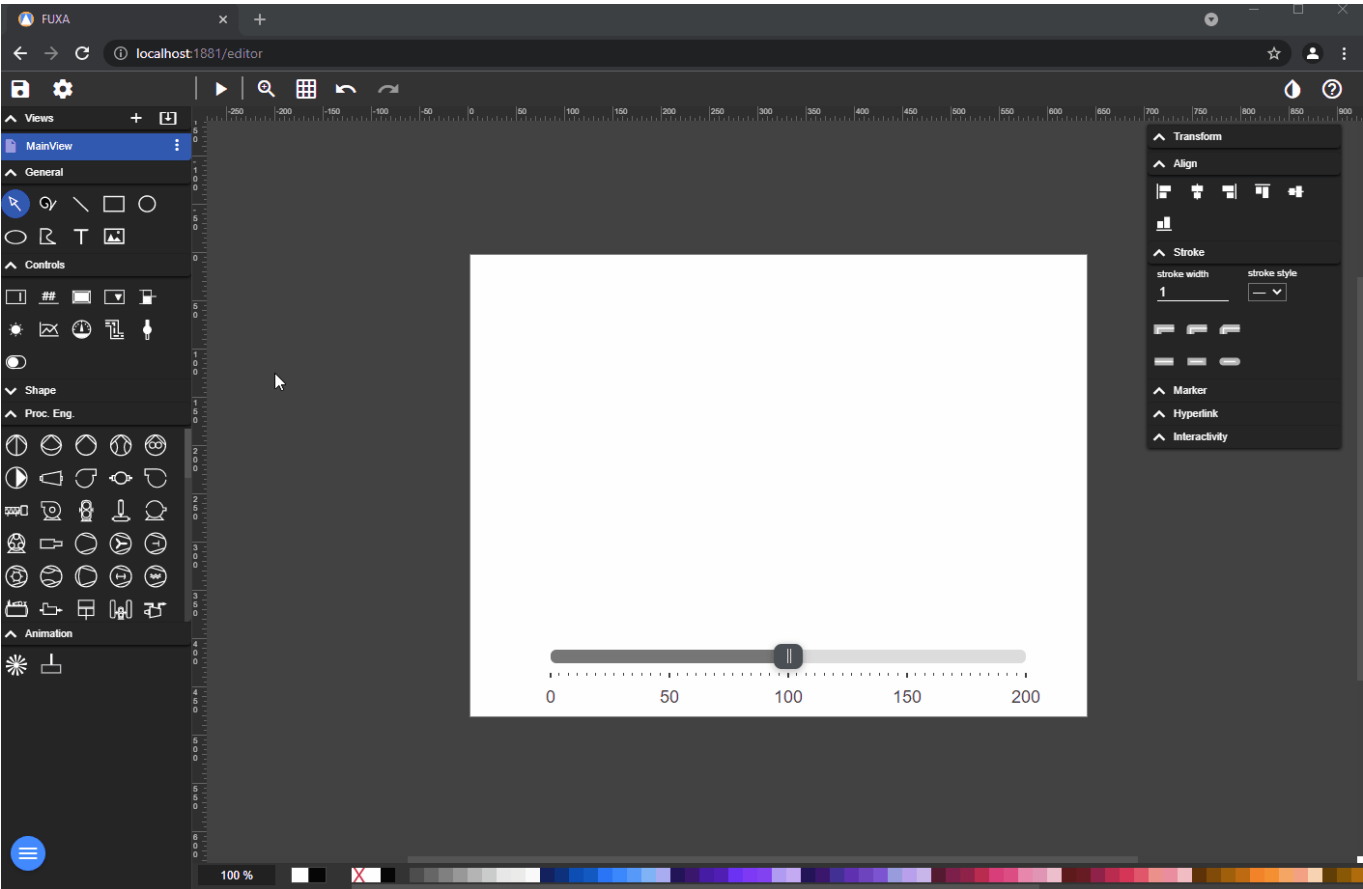
点击 "Property"。



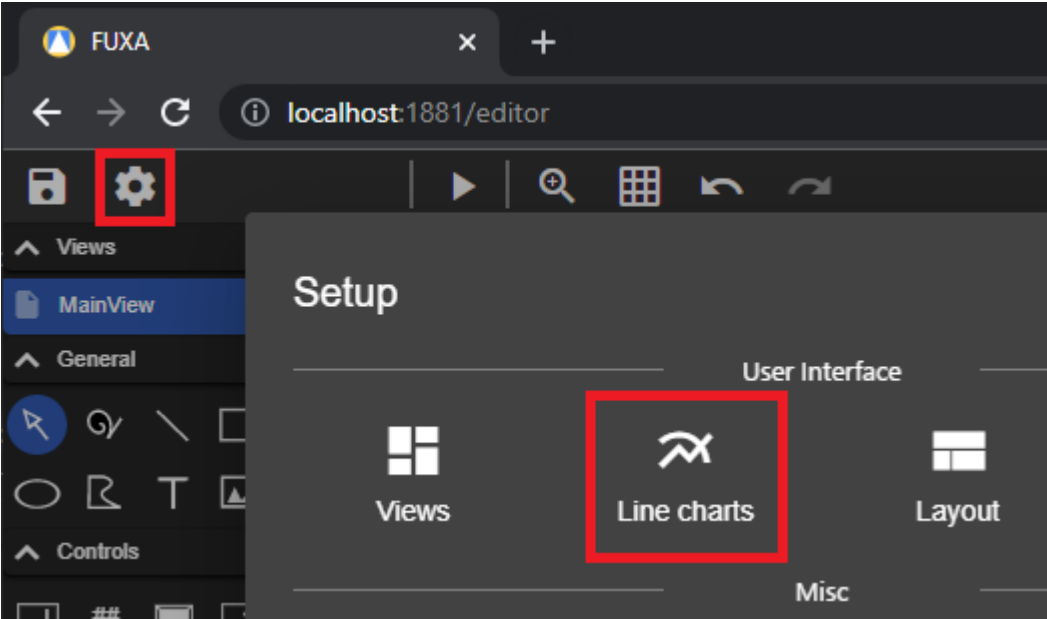
定义视图大小和背景。

8. 图表控件

要将 **图表** 控件添加到视图：在编辑器中选择视图。

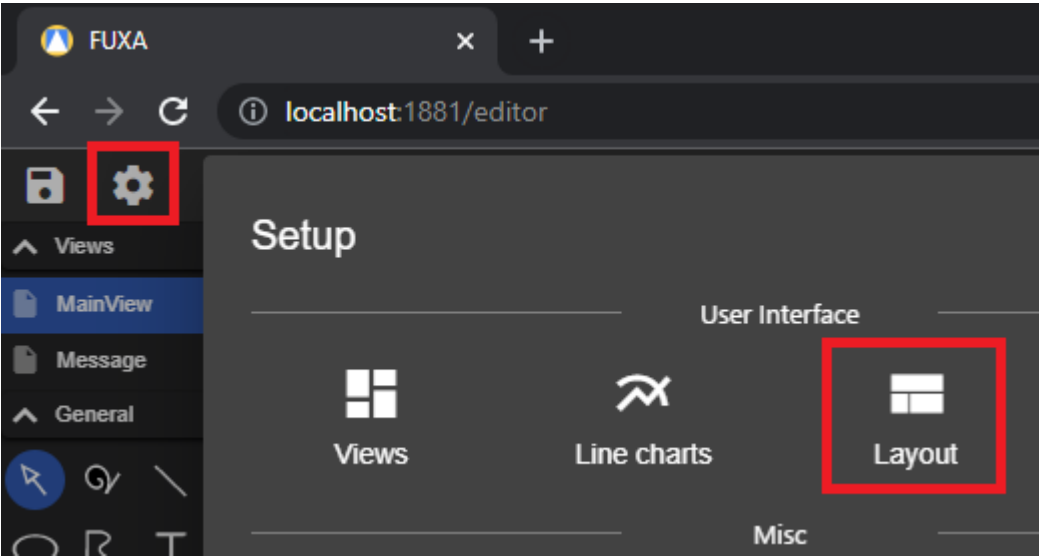


在 **折线图** 中，您可以定义附加到图表控件的图表。



9. UI 布局

要定义最终用户的布局，请在编辑器中进入 **Layout settings**。



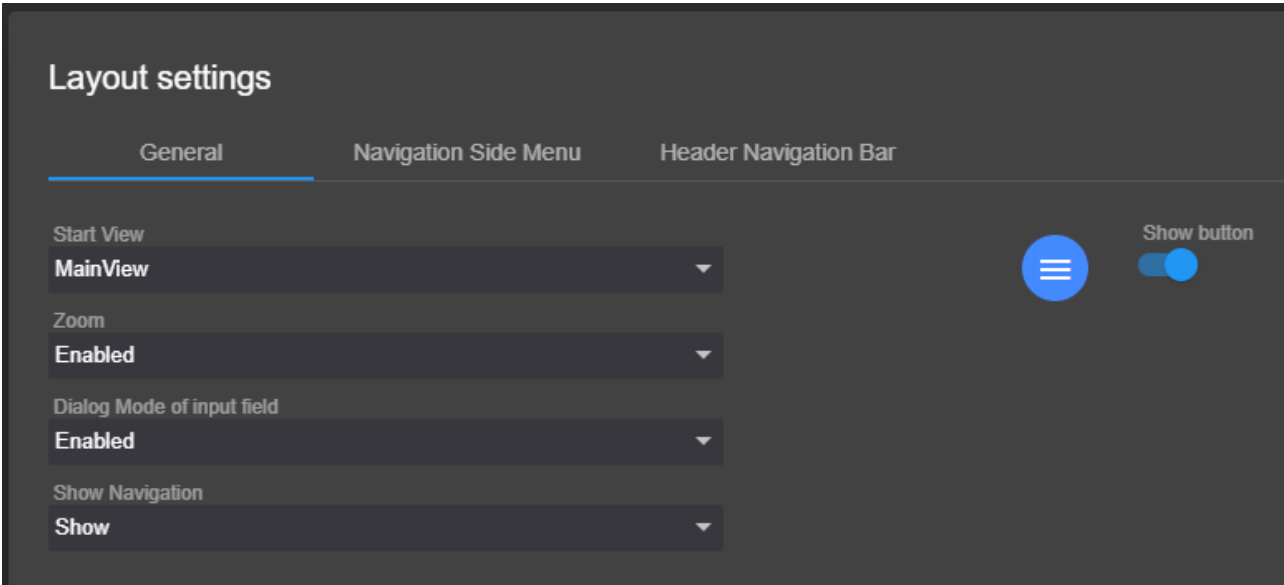
在 **常规** 选项卡中，您可以定义：

- **起始视图**，访问 <http://localhost:1881> 时显示的第一个视图
- **缩放**，如果启用，您可以使用鼠标滚轮缩放视图，并使用鼠标左按钮移动视图
- **输入字段的对话框模式**，如果启用，将在输入值控件时显示对话框

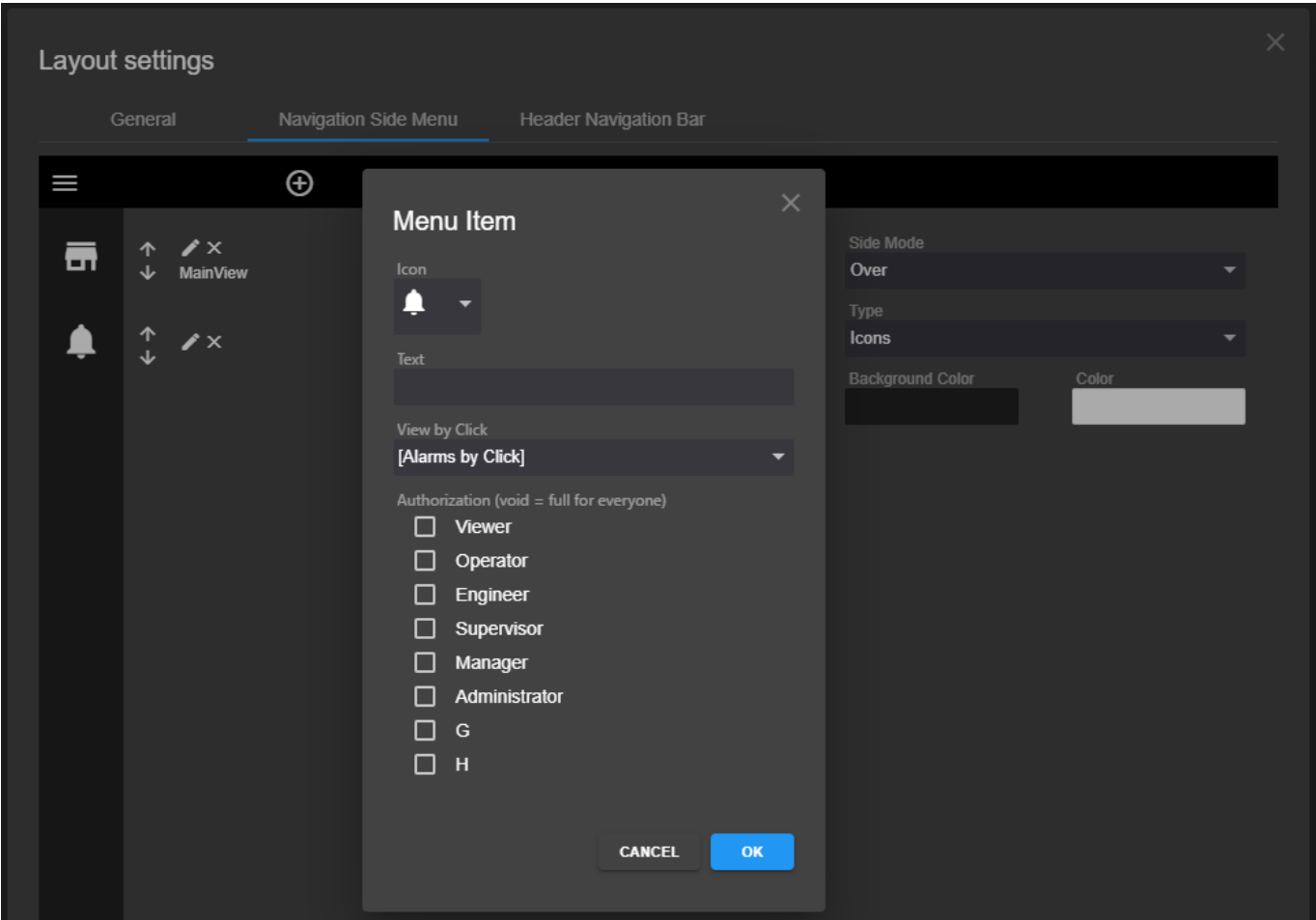


- **显示导航**，设置显示和隐藏标题栏和导航菜单

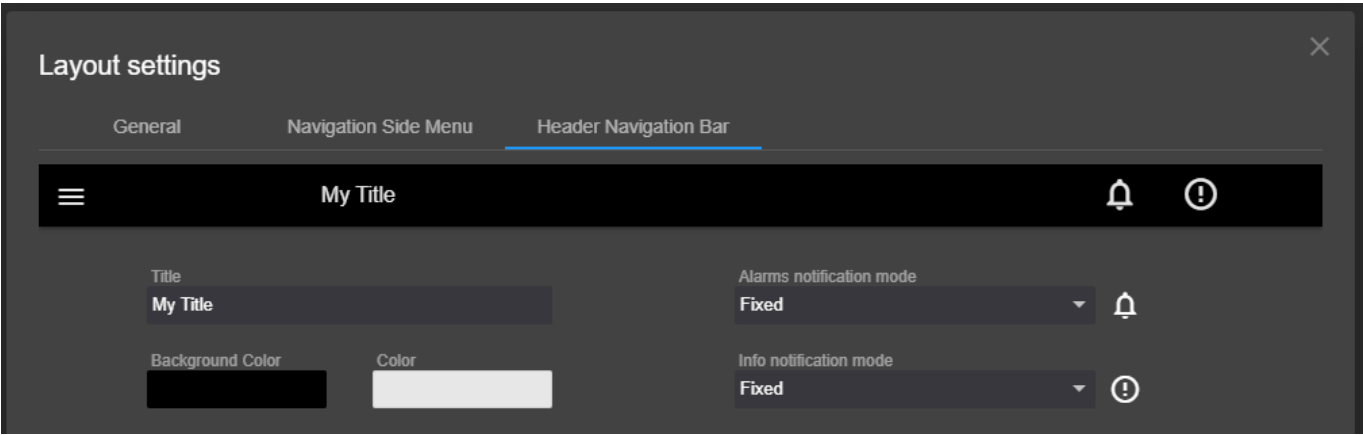
- 隐藏在编辑器和主页之间切换的开发按钮



在 **导航侧边菜单** 选项卡中，您可以定义菜单项和不同的样式属性。

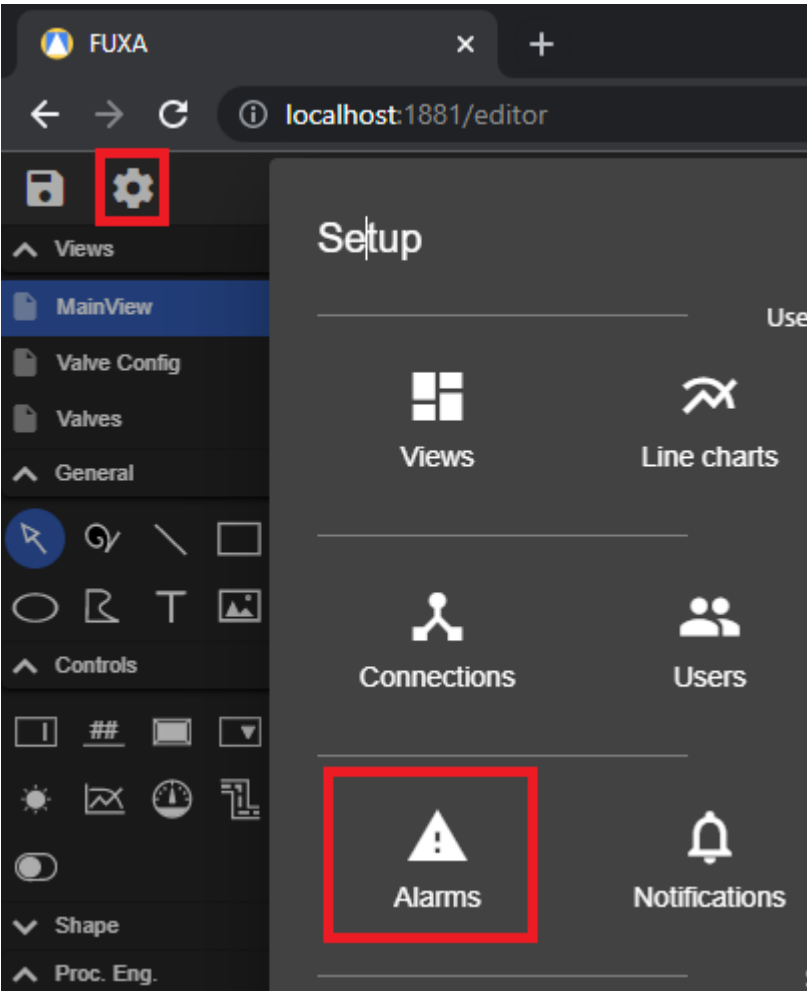


在 **标题导航栏** 选项卡中，您可以定义是否以及如何显示报警通知项，以及不同的样式属性。

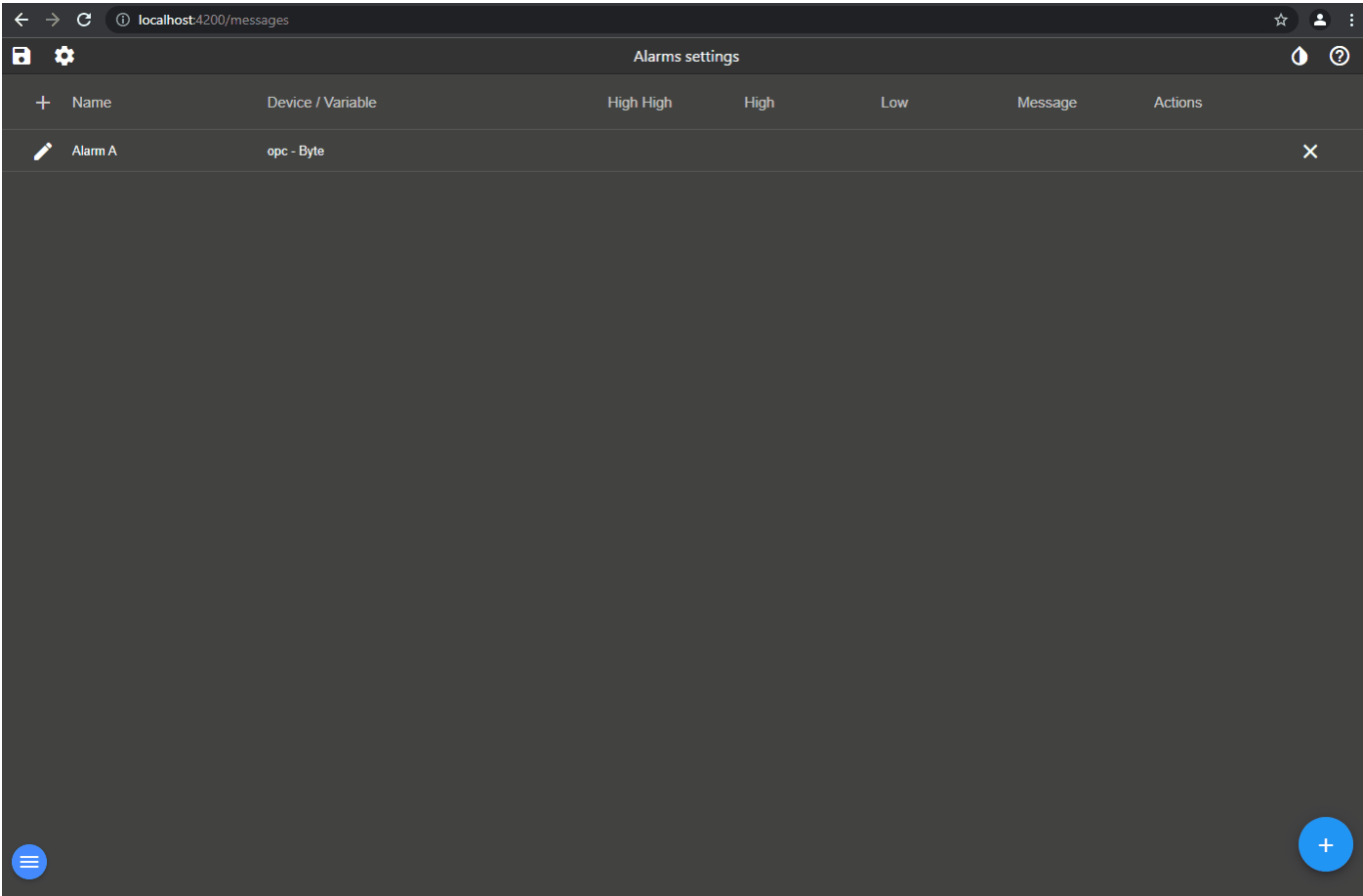


10. 报警设置

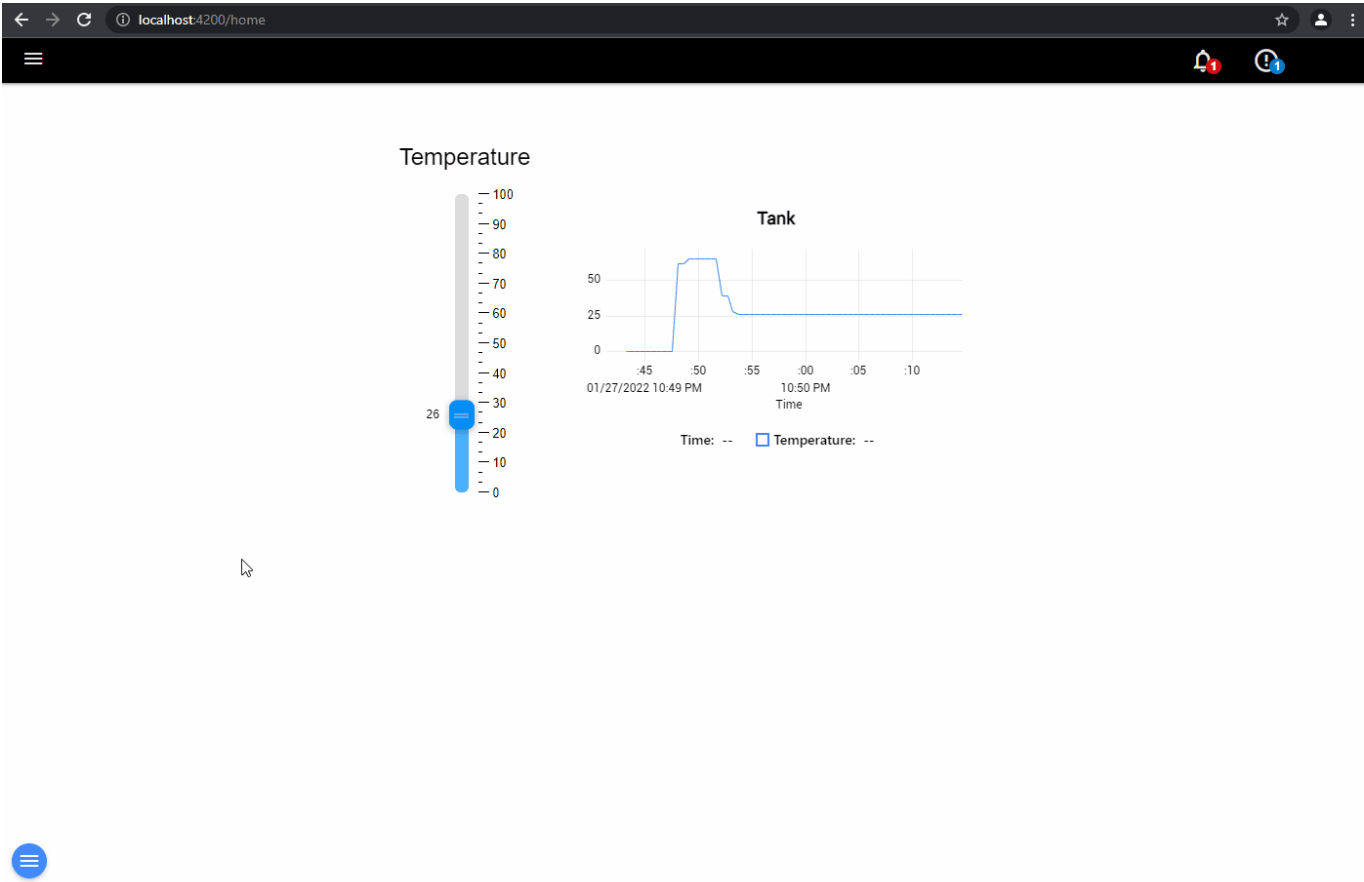
要在编辑器中配置报警，进入 **Alarms**。



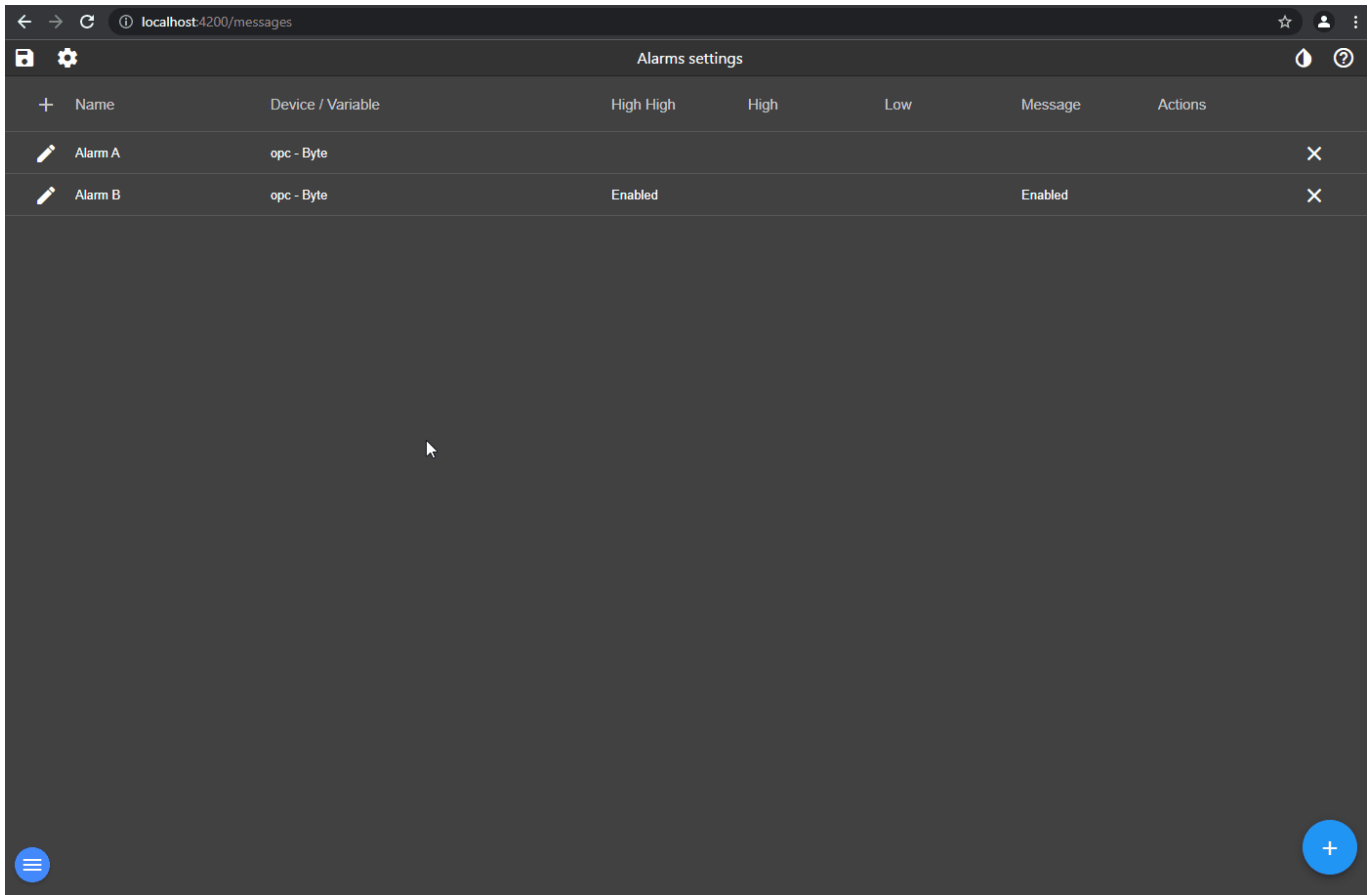
报警绑定到一个标签，您可以为此定义 4 种类型的条件，分别对应不同的级别（高高、高、低和消息）。



可以通过从菜单配置视图或在标题栏上按钮来显示活动报警和历史记录。 [布局设置](#)



还可以为每个报警配置多个操作，如显示弹出对话框或设置标签值。



11. 自定义图形

图形文件夹位于：`client/dist/assets/lib/svgeditor/shapes`（或在调试时为 `client/src/lib/svgeditor/shapes`），您可以通过创建新的 JavaScript 文件（最好从 `my-shapes.js` 复制）或编辑现有文件来添加新图形。

在 JavaScript 文件中，您可以更改以下内容：

```
var shapesGroupName = 'Shapes'; // 用于组织和分组图形，在编辑器菜单中显示为带展开/折叠的标签

var typeId = 'shapes'; // 用于标识图形类型，'shapes' 与 angular 组件 'ShapesComponent' 绑定
// 如果创建新类型，您还需要实现 angular 组件
```

在此数组中放置您的图形数据，图形对象具有以下属性：

```
var shapes = [{ name: 'diamond',
  ico: 'assets/lib/svgeditor/shapes/img/shape-diamond.svg',
  content:[{ id: '',
    type: 'path',
    attr: { d: 'M 20 0 L 40 20 L 20 40 L 0 20 Z' }
  }],...}]
```

```
// 'name': 图形类型的唯一标识
// 'ico': 在编辑器菜单中显示的图标路径
// 'content': svg 元素数组
// 'id': 元素 id, 如果您想进行动画处理, 请在 angular 组件中管理
// 'type': svg 元素类型 (path, text, ellipse, ...) 请参阅 svg 描述
// 'attr': 元素属性, 取决于类型
```

我使用 Inkscape 应用程序来设计图形。您可以使用 XML 编辑器查看元素和节点属性, 以便在 'type' 和 'attr' 中添加。

！您必须保留文件的其余内容

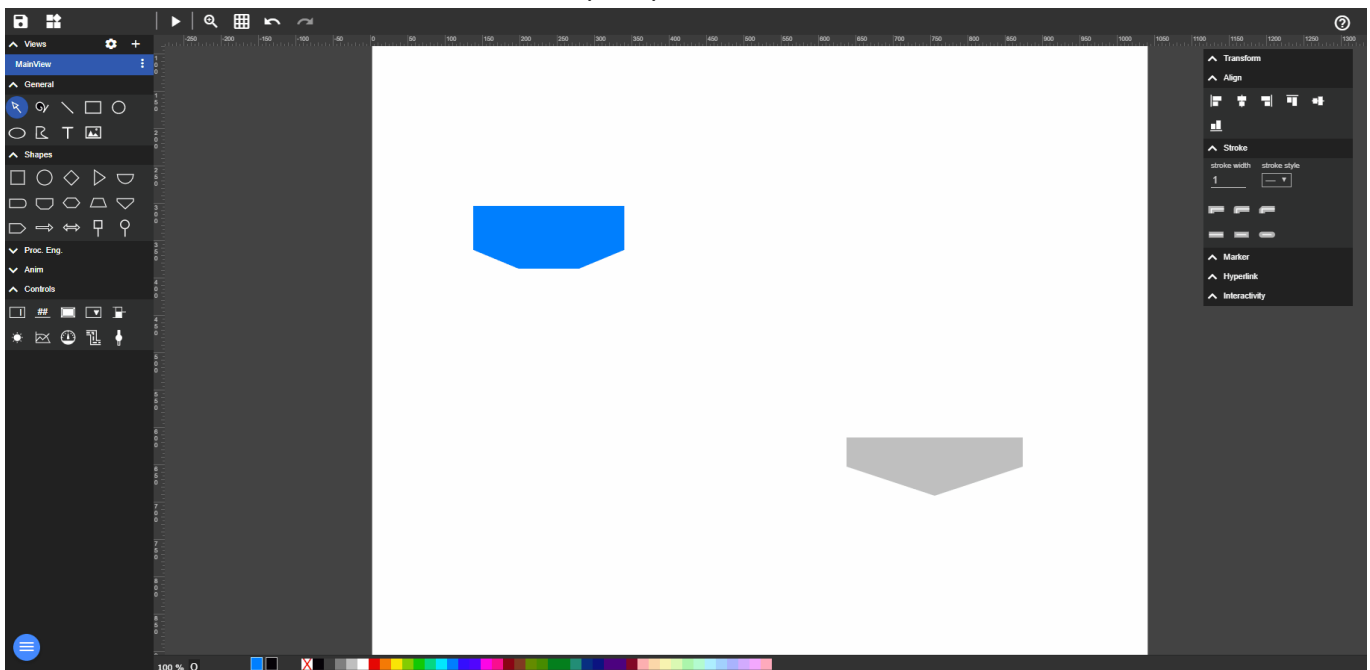
如果您创建了新的 JavaScript 图形文件, 您还需要将其添加到加载器, 然后在 `shapesLoader.js` 中添加您的文件名, 该文件将被动态加载 (请勿删除 'shapesLoader.js')

```
var shapesToLoad = ['my-shapes.js', 'your shape file name.js'];
```

如果您创建了新的图形类型, 例如定义自己的动画, 您还需要实现相应的 angular 组件。最好查看 'client/src/app/gauges/shapes/' 中的文件内容。

12. 管道动画

首先设计您的管道形状, 然后通过将设备标签 (变量) 绑定到动画来定义动作。



13. 调度器

调度器是 FUXA 中强大的基于时间的自动化系统, 允许您创建基于特定时间 (按星期几、每月几号和几月) 触发的事件。它提供计时器模式 (开/关期间) 和事件模式 (基于持续时间的触发), 具有完整的设备控制功能。

概述

调度器支持根据时间表自动控制设备和标签。主要功能包括：

- **按星期几调度**：为特定的日子（周一至周日）安排事件
- **月份模式**：按月份和日期组合进行高级调度
- **计时器模式**：定义开/关期间的开始和结束时间
- **事件模式**：触发特定持续时间的事件
- **设备操作**：执行其他操作，如设置值或运行脚本
- **主控制**：调度器作为设备标签的主控制器

调度模式

按星期几模式

默认调度模式允许您选择一周中特定的日子进行事件。

- **日期**：周一、周二、周三、周四、周五、周六、周日
- **时间范围**：为计时器模式定义开始和结束时间，或为事件模式定义开始时间和持续时间
- **视觉指示器**：选中的日期在调度显示中高亮显示

月份模式

高级调度，结合月份和日期选择以实现精确控制。

- **月份**：一月至十二月
- **日期**：1日至31日
- **重要说明**：
 - 如果某月少于 31 天（例如 2 月有 28 天），31 日安排的事件将不执行
 - Node.js 调度器不支持"月末"功能
 - 事件仅在有效日期组合上触发

设备和标签关系

设备绑定

每个调度绑定到特定设备并控制其关联的标签：

- **设备选择**：从项目中可用的设备中选择
- **标签控制**：调度器直接控制设备的主标签
- **主权限**：调度器始终优先于手动标签更改

设备操作

除了控制主设备标签外，您还可以配置其他操作：

- **设置值**：将其他设备标签设置为特定值
- **运行脚本**：执行自定义 JavaScript 函数
- **触发点**：操作在开始和结束事件时执行

设备操作在调度器属性中配置，提供超越基本开/关控制的扩展自动化功能。

授权

调度器实施双层授权系统：

主授权

- **自动分配**：调度的设备自动接收主授权级别
- **最低要求**：用户必须至少具有主授权才能与调度的设备交互
- **覆盖能力**：主授权允许调度器在活动事件期间覆盖手动标签更改

按设备授权

- **额外安全**：单个设备可以具有高于主要求的授权级别
- **细粒度控制**：不同设备可能需要不同的权限级别（例如，关键设备可能需要管理员访问）
- **分层安全**：主授权提供基本访问，而按设备授权添加设备特定限制

添加/编辑调度表单

基本设置

- **设备**：选择要控制的目标设备
- **开始时间**：事件应该开始的时间（HH:MM 格式）
- **模式切换**：在计时器模式和事件模式之间切换

计时器模式 vs 事件模式

计时器模式

- **结束时间**：定义事件何时停止
- **标签行为**：标签从开始时间到结束时间保持开启
- **用例**：常规开/关期间（例如，上午 8:00 至下午 6:00）

事件模式

- **持续时间**：设置事件应持续多长时间
- **持续时间单位**：小时、分钟、秒
- **标签行为**：标签在指定持续时间内开启，然后自动关闭
- **用例**：定时触发（例如，运行泵 30 秒）

循环 vs 一次性事件

- **循环**：事件根据调度重复（默认）
- **一次性**：事件执行一次后自动删除
- **自动清理**：非循环事件完成后自动删除以防止累积

月份模式配置

启用月份模式时：

- **月份选择**：选择事件应运行的月份
- **日期选择**：在这些月份内选择特定日期
- **日历视图**：点击日历按钮查看调度的可视化表示

数据存储

数据库结构

调度存储在项目数据库中，结构如下：

```
{
  "schedules": {
    "Device Name": [
      {
        "id": "unique_schedule_id",
        "startTime": "08:00",
        "endTime": "18:00",
        "days": [false, true, true, true, true, true, false],
        "months": [false, false, false, false, false, false, false, false, false,
true, false, false],
        "daysOfMonth": [false, false, false, false, true, false, false, false,
false, false, false, false, false, false, false, false,
false, false, false, false, false, false, false, false,
false],
        "monthMode": true,
        "recurring": true,
        "eventMode": false,
        "duration": 0,
        "deviceName": "Device Name",
        "variableId": "tag_id"
      }
    ]
  },
  "settings": {
    "deviceActions": [...]
  }
}
```

关键字段

- **days**：周一至周日的布尔数组（7 个元素）
- **months**：1月至12月的布尔数组（12 个元素）
- **daysOfMonth**：1日至31日的布尔数组（31 个元素）
- **monthMode**：为 true 时启用基于月份的调度
- **eventMode**：为 true 时启用基于持续时间的事件
- **recurring**：控制事件是否重复或执行一次

使用示例

日常泵调度

- 设备：水泵
- 日期：周一至周五
- 开始：08:00，结束：18:00
- 模式：计时器模式
- 结果：工作日内泵在工作时间运行

月度维护

- 设备：维护阀
- 月份：一月、四月、七月、十月
- 日期：每月15日
- 开始：09:00，持续时间：2小时
- 模式：事件模式，非循环
- 结果：按季度计划维护阀打开

节日照明

- 设备：节日灯
- 月份：十二月
- 日期：1日至31日
- 开始：17:00，结束：23:00
- 模式：计时器模式
- 结果：十二月每晚灯运行

最佳实践

1. **测试调度**：在开发期间始终测试新调度
2. **月份限制**：避免安排无效的日期组合
3. **设备操作**：将设备操作用于复杂的自动化逻辑
4. **时区**：注意服务器的时区安排
5. **资源管理**：非循环事件自动清理以防止累积

故障排除

常见问题

- **事件未触发**：检查月份/日期组合的有效性
- **标签未更改**：验证设备连接和权限
- **操作未执行**：检查设备操作配置
- **调度未保存**：确保正确的权限和数据库访问

调试信息

调度器日志包含以下详细信息：

- 调度加载和解析
- 带时间戳的事件触发

- 标签写操作
- 设备操作执行
- 月份模式日期验证

14. Node-RED 集成

概述

FUXA 包含完整的 Node-RED 集成，允许您创建与 SCADA 系统交互的强大自动化流程。

设置和安装

FUXA 中的 Node-RED

Node-RED 自动随 FUXA 一起包含，无需额外安装。通过 FUXA 设置菜单访问它。

安装 Dashboard 2

如果需要，Dashboard 2 必须单独安装：

1. 打开 Node-RED 编辑器（通过 FUXA 设置）
2. 点击菜单 (三) → **Manage Palette**
3. 搜索：@flowfuse/node-red-dashboard
4. 点击 **Install**
5. 重启 Node-RED/FUXA

所需依赖项

- **@flowfuse/node-red-dashboard**：用于现代仪表板创建
- **node-red-contrib-fuxa**：自动包含（提供 FUXA 集成节点）

配置

无需额外配置。Node-RED 自动连接到 FUXA 的运行时环境。

安全和访问模式

当启用 FUXA 安全时，可以配置 Node-RED 访问。

设置： nodeRedAuthMode

- **secure**（默认）：Node-RED 编辑器和管理 API 需要身份验证（JWT 或 API 密钥）。
- **legacy-open**：Node-RED 保持开放（无身份验证）以保留传统行为。

注意：

- 更改在服务器重启后生效。
- 如果 **secureEnabled=true** 和 **nodeRedAuthMode=secure**，外部系统可以使用 API 密钥头访问 Node-RED HTTP 端点：**x-api-key: <api-key>** API 密钥在 FUXA 中的 **Editor** → **Setup** → **API Keys** 下管理。

访问 Node-RED

Node-RED 编辑器

通过 FUXA 设置访问 Node-RED 流程编辑器：

1. 点击 FUXA 中的 **Settings** 按钮
2. 导航到 **Node-RED** 部分
3. 点击 **Open Node-RED Editor**

这将在新窗口/标签中打开 Node-RED 流程编辑器。

Dashboard 2

注意：默认不安装 Dashboard 2，必须单独安装。

安装

1. 在 Node-RED 中，转到 **Menu → Manage Palette**
2. 搜索并安装：[@flowfuse/node-red-dashboard](#)
3. 重启 Node-RED/FUXA

访问 Dashboard 2

安装后，查看您的 Node-RED 仪表板：

嵌入 FUXA 视图中：将 iframe 组件添加到任何 FUXA 视图

在 FUXA 视图中嵌入仪表板

1. 在编辑器中打开您的 FUXA 项目
2. 将 iframe 组件添加到您的视图
3. 将 URL 设置为：[/dashboard](#)
4. 根据需要配置 iframe 大小和属性
5. 如果只想显示单个页面，可以调整仪表板设置并使用 [/dashboard/page1](#) 或您的页面名称

重要：在尝试访问 [/dashboard](#) 之前必须安装 Dashboard 2

FUXA 贡献节点

FUXA contrib 包提供用于 SCADA 集成的专用节点，此包设计为仅与 FUXA Node-Red 集成配合使用，不能在独立 Node-Red 安装中工作。

标签节点

get-tag

目的：获取 FUXA 标签的当前值

参数：

- **Name**: 可选的节点名称用于识别
- **Tag**: 标签名称 (从 FUXA 设备下拉填充)

输入: 任何消息 (触发标签读取) **输出**: `msg.payload` 包含标签值

set-tag

目的: 向 FUXA 标签写入值

参数:

- **Name**: 可选的节点名称用于识别
- **Tag**: 标签名称 (从 FUXA 设备下拉填充)

输入: `msg.payload` 包含要写入的值 **输出**: `msg.payload` 包含写入的值

get-tag-change

目的: 基于设备轮询事件监控 FUXA 标签的值变化

参数:

- **Name**: 可选的节点名称用于识别
- **Tag**: 标签名称 (从 FUXA 设备下拉填充)

输入: 无 (事件驱动) **输出**: `msg.payload` 包含变化时的新标签值

工作原理: 此节点订阅 FUXA 的设备轮询事件。当设备被轮询且标签值自上次轮询以来已更改时, 节点输出带有新值的消息。时机取决于设备的轮询间隔 (通常为 1-5 秒)。

注意: 此节点仅在标签值实际更改时输出消息, 而不是在每个轮询周期。它有效地监控变化, 无需手动轮询。

get-tag-id

目的: 通过名称获取标签的内部 ID

参数:

- **Name**: 可选的节点名称用于识别
- **Tag**: 标签名称 (从 FUXA 设备下拉填充)

输入: 任何消息 **输出**: `msg.payload` 包含标签 ID

get-historical-tags

目的: 同时检索多个标签的历史数据

参数:

- **Name**: 可选的节点名称用于识别
- **Tags**: 逗号分隔的标签名称列表 (例如 "temp1,temp2,pressure")
- **From Time**: 历史时段的开始 (datetime-local 选择器)

- **To Time:** 历史时段的结束 (datetime-local 选择器)

输入: 任何消息或使用 `msg.tags`、`msg.from`、`msg.to` 覆盖 **输出:** `msg.payload` 包含历史数据数组

get-tag-daq-settings

目的: 获取标签的 DAQ (数据采集) 设置

参数:

- **Name:** 可选的节点名称用于识别
- **Tag:** 标签名称 (从 FUXA 设备下拉填充)

输入: 任何消息 **输出:** `msg.payload` 包含 DAQ 设置对象

set-tag-daq-settings

目的: 配置标签的 DAQ (数据采集) 设置

参数:

- **Name:** 可选的节点名称用于识别
- **Tag:** 标签名称 (从 FUXA 设备下拉填充)
- **Enabled:** 启用/禁用此标签 DAQ 的复选框
- **Interval:** 采样间隔 (毫秒)
- **Deadband:** 数据更新的最小变化阈值

输入: 任何消息或使用 `msg.tag`、`msg.enabled`、`msg.interval`、`msg.deadband` 覆盖 **输出:** `msg.payload` 包含更新后的设置

设备节点

enable-device

目的: 启用或禁用设备连接

参数:

- **Name:** 可选的节点名称用于识别
- **Device Name:** 设备名称 (从 FUXA 设备下拉填充)
- **Enabled:** 启用/禁用设备的复选框

输入: 任何消息或使用 `msg.deviceName`、`msg.enabled` 覆盖 **输出:** `msg.payload` 包含操作结果

get-device

目的: 获取 FUXA 设备的信息

参数:

- **Name:** 可选的节点名称用于识别
- **Device Name:** 设备名称 (从 FUXA 设备下拉填充)

- **Include Tags:** 复选框以在响应中包含标签信息

输入: 任何消息或使用 `msg.deviceName`、`msg.includeTags` 覆盖 **输出:** `msg.payload` 包含设备信息对象

Include Tags: 选中时, 响应包含与设备关联的所有标签

get-device-property

目的: 从设备获取属性值

参数:

- **Name:** 可选的节点名称用于识别
- **Device Name:** 设备名称 (从 FUXA 设备下拉填充)
- **Property:** 要检索的属性名称

输入: 任何消息或使用 `msg.deviceName`、`msg.property` 覆盖 **输出:** `msg.payload` 包含属性值

属性名称: 常见属性包括 "status"、"connected"、"lastError" 等

set-device-property

目的: 在设备上设置属性值

参数:

- **Name:** 可选的节点名称用于识别
- **Device Name:** 设备名称 (从 FUXA 设备下拉填充)
- **Property:** 要设置的属性名称
- **Value:** 要为属性设置的值

输入: `msg.payload` 包含值或使用 `msg.deviceName`、`msg.property`、`msg.value` 覆盖 **输出:** `msg.payload` 包含设置的值

报警节点

get-alarms

目的: 获取当前活动报警

参数:

- **Name:** 可选的节点名称用于识别

输入: 任何消息 **输出:** `msg.payload` 包含活动报警数组

get-history-alarms

目的: 获取历史报警数据

参数:

- **Name**: 可选的节点名称用于识别
- **Start Time**: 历史时段的开始 (datetime-local 选择器)
- **End Time**: 历史时段的结束 (datetime-local 选择器)

输入: 任何消息或使用 `msg.startTime` 和 `msg.endTime` 覆盖 **输出**: `msg.payload` 包含历史报警数组

ack-alarm

目的: 确认 FUXA 中的报警

参数:

- **Name**: 可选的节点名称用于识别
- **Alarm Name**: 报警名称 (从 FUXA 报警下拉填充)
- **Types**: 逗号分隔的报警类型 (可选)

输入: 任何消息或使用 `msg.alarmName`、`msg.types` 覆盖 **输出**: `msg.payload` 包含确认结果

视图节点

set-view

目的: 更改 FUXA 中的当前视图

参数:

- **Name**: 可选的节点名称用于识别
- **View Name**: 视图名称 (从 FUXA 视图下拉填充)

输入: 任何消息或使用 `msg.viewName` 覆盖 **输出**: `msg.payload` 包含操作结果

open-card

目的: 打开 FUXA 中的特定卡片/对话框

参数:

- **Name**: 可选的节点名称用于识别
- **Card**: 卡片名称 (从 FUXA 卡片下拉填充)

输入: 任何消息或使用 `msg.cardName` 覆盖 **输出**: `msg.payload` 包含操作结果

脚本节点

execute-script

目的: 执行 FUXA 脚本

参数:

- **Name**: 可选的节点名称用于识别
- **Script**: 脚本名称 (从 FUXA 脚本下拉填充)

输入：任何消息或使用 `msg.scriptName` 覆盖 **输出：**`msg.payload` 包含执行结果

DAQ 节点

get-daq

目的：获取单个标签的数据采集（DAQ）数据

参数：

- **Name：**可选的节点名称用于识别
- **Tag：**单个标签名称（从 FUXA 设备下拉填充）
- **From Time：**DAQ 时段的开始（datetime-local 选择器）
- **To Time：**DAQ 时段的结束（datetime-local 选择器）

输入：任何消息或使用 `msg.from`、`msg.to` 覆盖 **输出：**`msg.payload` 包含单个标签的 DAQ 数据

仅单个标签：与 get-historical-tags 不同，此节点一次仅处理一个标签。对于多个标签，请使用单独的 get-daq 节点或 get-historical-tags。

事件节点

emit-event

目的：通过 FUXA 的事件系统发出自定义事件，用于系统间通信和自动化触发

参数：

- **Name：**可选的节点名称用于识别
- **Event Type：**自定义事件名称/类型（例如 "machine-fault"、"production-complete"、"maintenance-required"）

输入：`msg.payload` 包含事件数据（任何格式：字符串、数字、对象、数组） **输出：**`msg.payload` 保持不变（传递通过）

事件类型和使用方式：

系统事件（FUXA 内置事件）：

- `device-status` - 设备连接状态更改
- `device-property` - 设备属性更新
- `device-values` - 标签值更改
- `alarms-status` - 报警状态更改
- `script-console` - 脚本控制台输出
- `heartbeat` - 系统心跳/存活信号

自定义事件（用户定义）：

- `production-start` - 生产线启动
- `production-stop` - 生产线停止
- `quality-alert` - 质量控制警报

- `maintenance-due` - 设备维护需求
- `operator-login` - 操作员身份验证
- `batch-complete` - 生产批次完成

send-message

目的：通过 FUXA 的通知系统发送电子邮件通知或消息

参数：

- **Name：**可选的节点名称用于识别
- **Address：**收件人电子邮件地址
- **Subject：**电子邮件主题行
- **Message：**默认消息内容（可覆盖）

输入：`msg.payload` 可以包含消息数据或使用 `msg.address`、`msg.subject`、`msg.message` 覆盖 **输出：**`msg.payload` 包含发送结果

数据格式

标签值

标签值可以是：

- **数字：**`25.3`、`100`、`0`
- **字符串：**`"Running"`、`"Stopped"`
- **布尔值：**`true`、`false`
- **对象：**复杂数据结构

时间戳

所有时间戳使用 ISO 8601 格式：

```
"2025-01-01T10:30:00.000Z"
```

故障排除

仪表板未加载

问题：`/dashboard` 显示错误或空白页

解决方案：安装 Dashboard 2

1. 打开 Node-RED 编辑器
2. 转到 **Manage Palette**
3. 安装 `@flowfuse/node-red-dashboard`
4. 重启 Node-RED/FUXA

FUXA 节点未出现

问题：FUXA contrib 节点在面板中不可见

解决方案：检查 Node-RED 日志中是否有错误。FUXA 启动 Node-RED 时节点会自动注册。

下拉列表未填充

问题：标签/设备下拉列表为空

解决方案：确保您在 FUXA 中配置了设备和标签。下拉列表从您的 FUXA 项目数据填充。

事件不工作

问题：发出的事件未触发预期行为

解决方案：检查其他 FUXA 组件（脚本、视图）是否正在侦听正确的事件类型。事件区分大小写。

电子邮件通知未发送

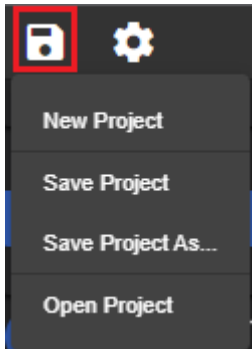
问题：send-message 节点未发送电子邮件

解决方案：在 FUXA 服务器配置中配置 SMTP 设置。检查 FUXA 日志中的 SMTP 连接错误。

15. 项目保存/加载

创建项目

您可以创建一个新项目，当前项目将被新的空项目覆盖（只有一个空的 MainView）。



保存项目

在每次更改设置后（如编辑 'Device'、'Tag'、'Charts' 等的对话框确认），项目将自动保存在内部数据库中。视图将通过选择另一个或离开编辑器来自动保存。

通过 **Save Project** 您可以强制执行内部保存过程。通过 **Save Project As...** 您可以将整个项目导出为 JSON 格式的文件（MyProject.json），可用于制作项目备份。

打开项目

您可以从导出的文件（MyProject.json）中打开项目。

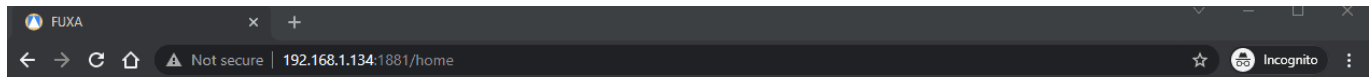
16. 事件配置

在图形和按钮控件中，您可以配置鼠标 **事件** 点击、mouseDown、mouseUp 来执行任务。

- [打开页面](#)
- [打开卡片](#)
- [打开对话框](#)
- [打开 iframe](#)
- [打开窗口](#)
- [设置值](#)
- [切换值](#)
- [从输入设置并关闭](#)

打开页面

用于在**主**窗口中显示视图。



打开卡片

用于将视图显示为弹出窗口。**卡片**将在事件发生时显示在鼠标旁边，可以同时显示多个卡片。相同的**卡片**例如可用于显示多个泵的值。



Open Card



打开对话框

用于将视图显示为对话框，通常用于配置值。**对话框**将显示在屏幕顶部。相同的**对话框**例如可用于配置多个泵的值。

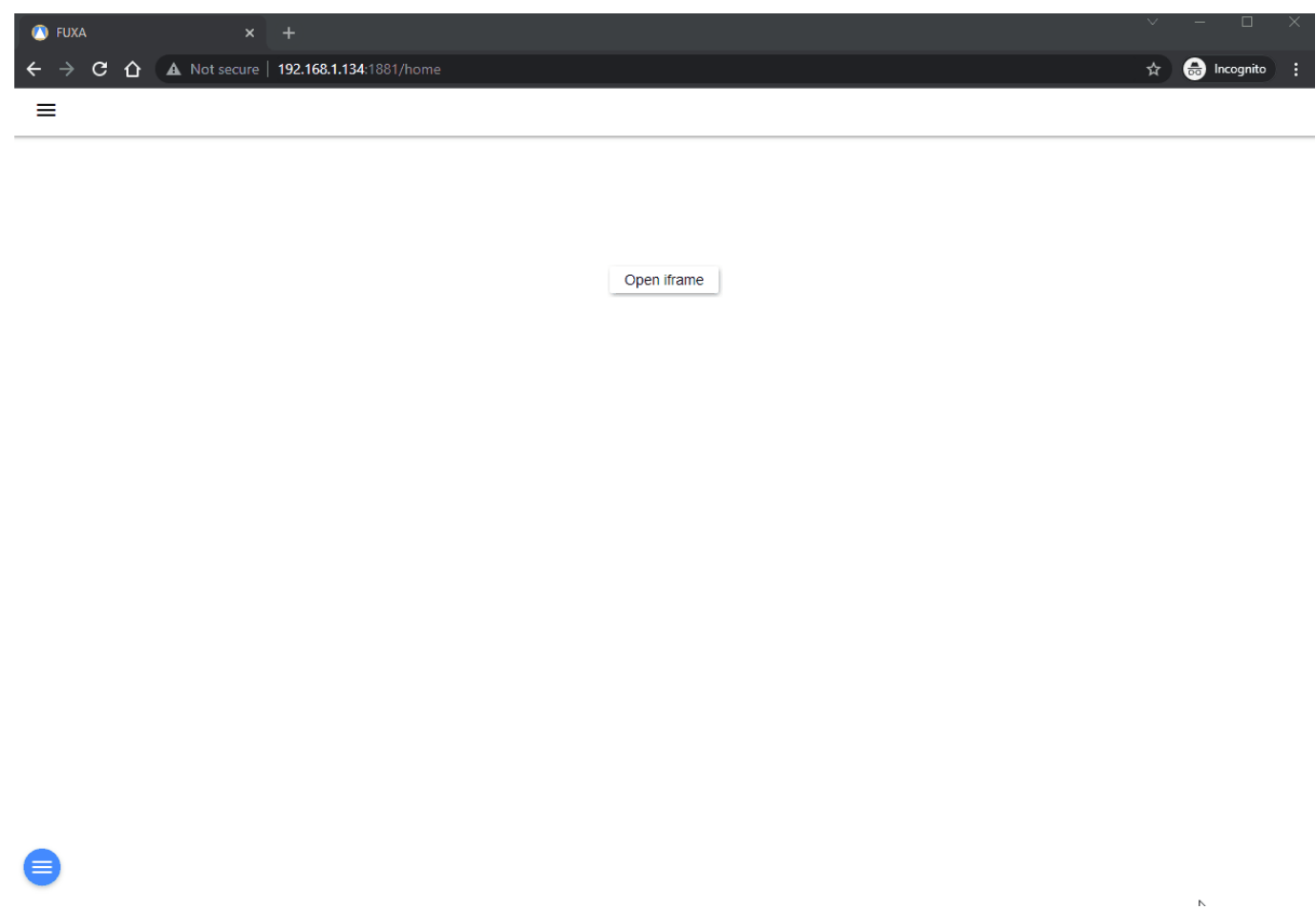


Open Dialog



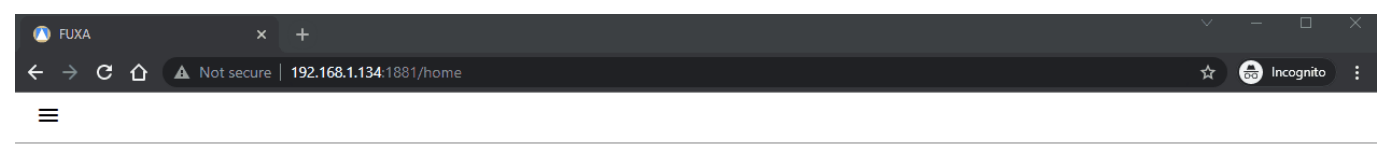
打开 iframe

用于将窗口作为 **iframe**（来自外部来源的嵌入 HTML 文档）打开。您可以定义窗口大小和缩放比例。



打开窗口

用于在新浏览器中打开窗口。您可以定义窗口大小。

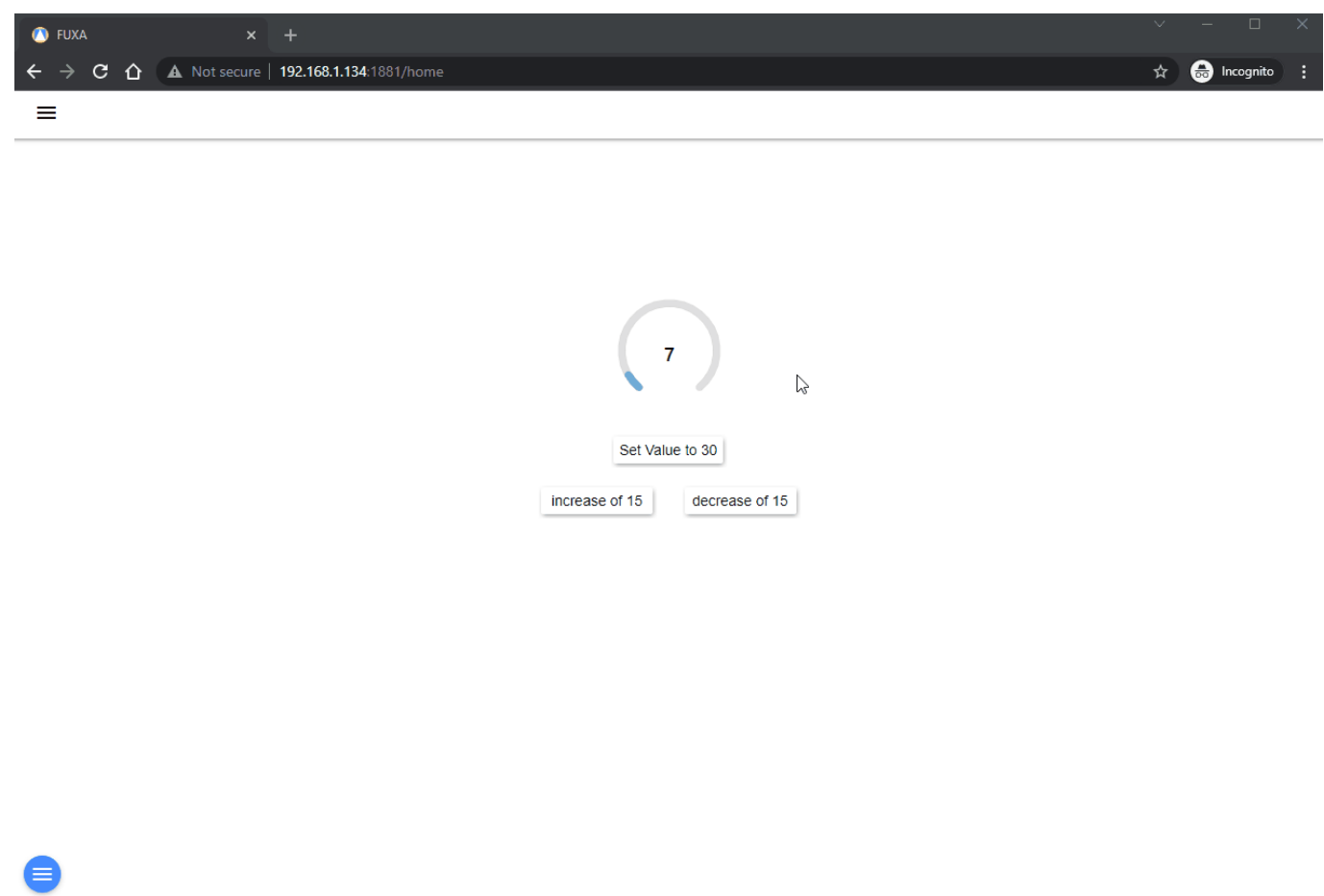


Open Window



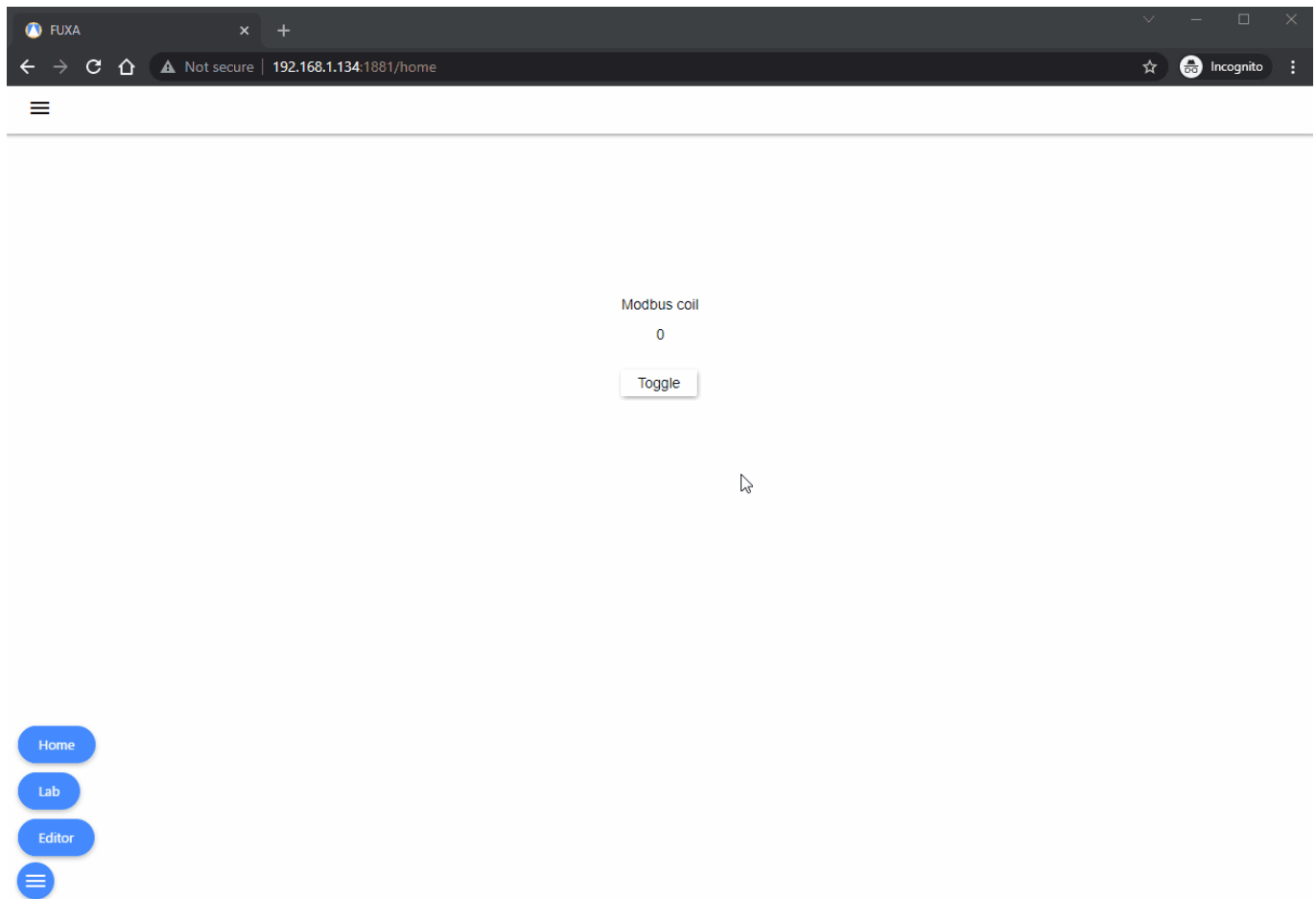
设置值

用于设置标签值或增加和减少当前值。



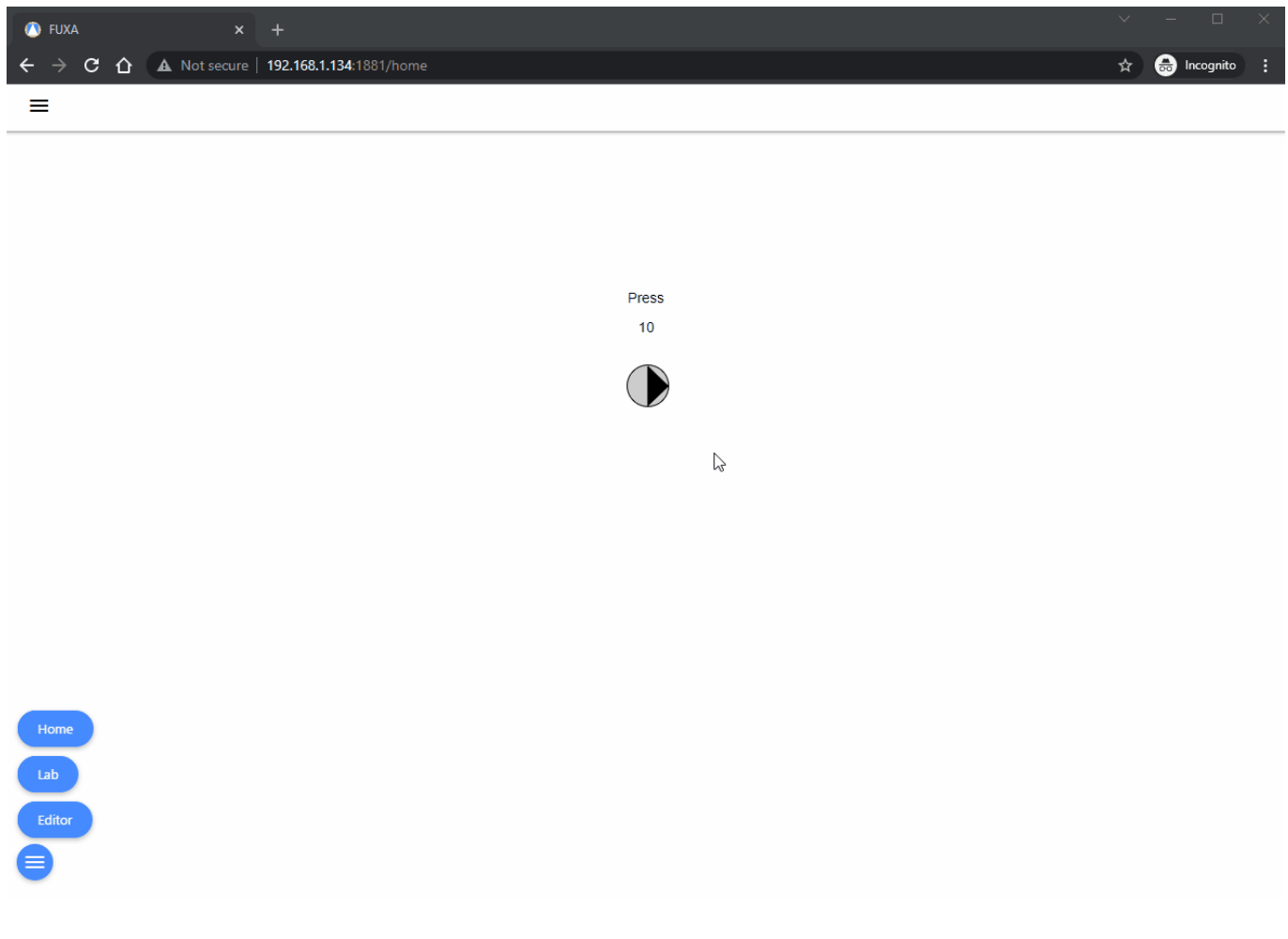
切换值

用于切换标签值 1/0（如果为 1 则设为 0，如果为 0 则设为 1）。



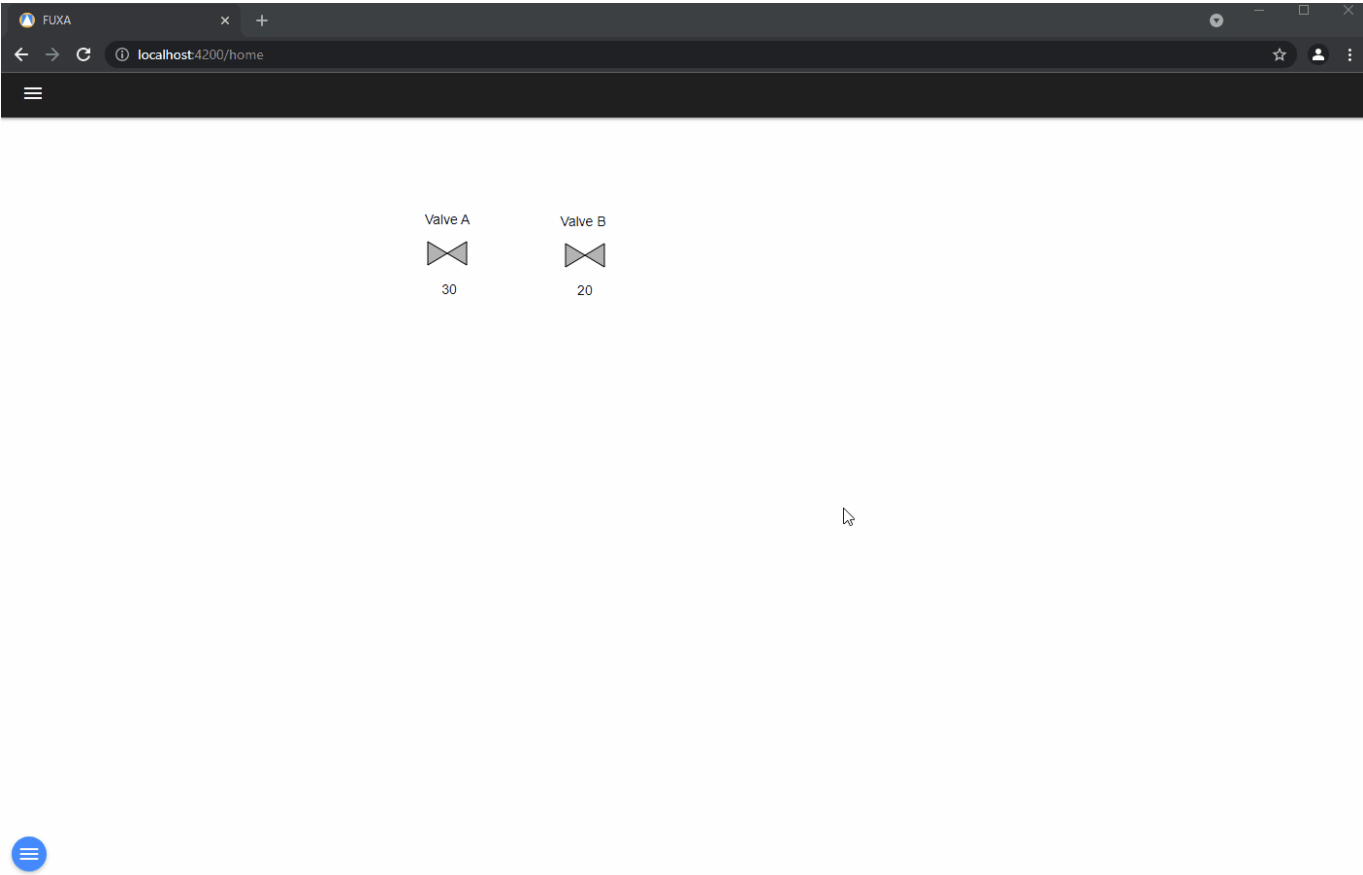
从输入设置并关闭

组合使用可通过带确认的对话框设置一些值。

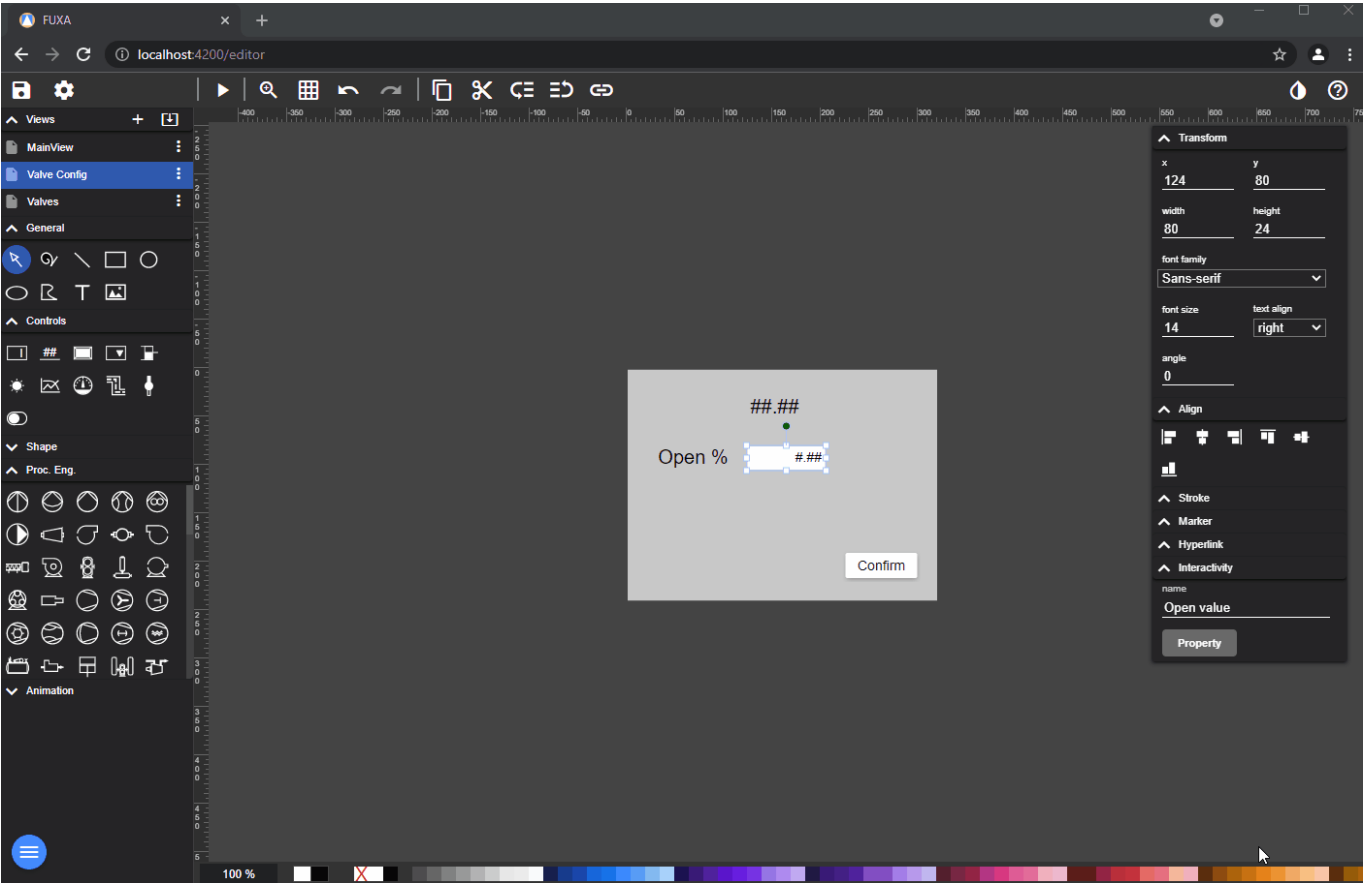


17. 复用视图

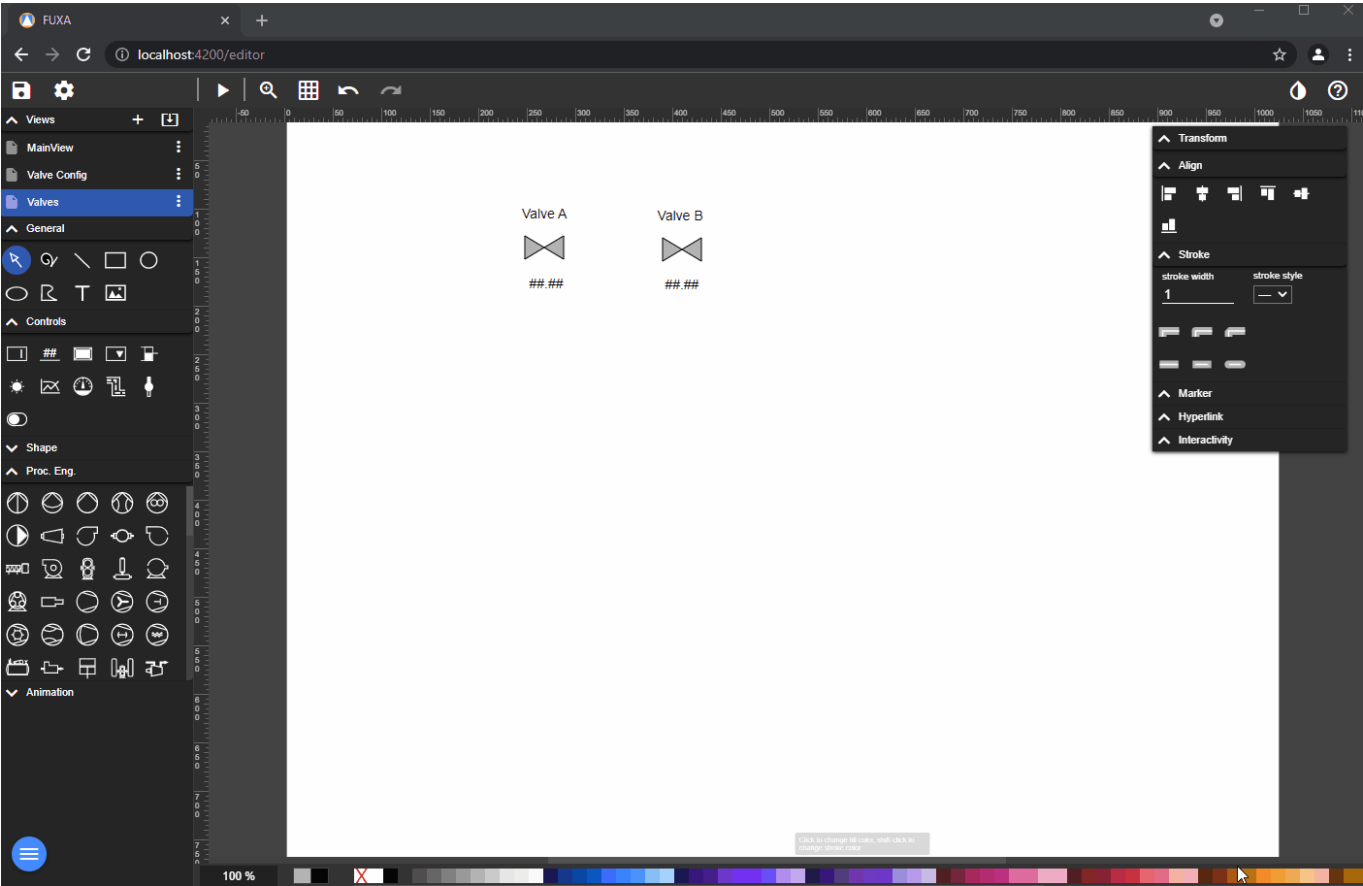
您可以为重复组件（如泵和阀门）复用相同的 **视图**。



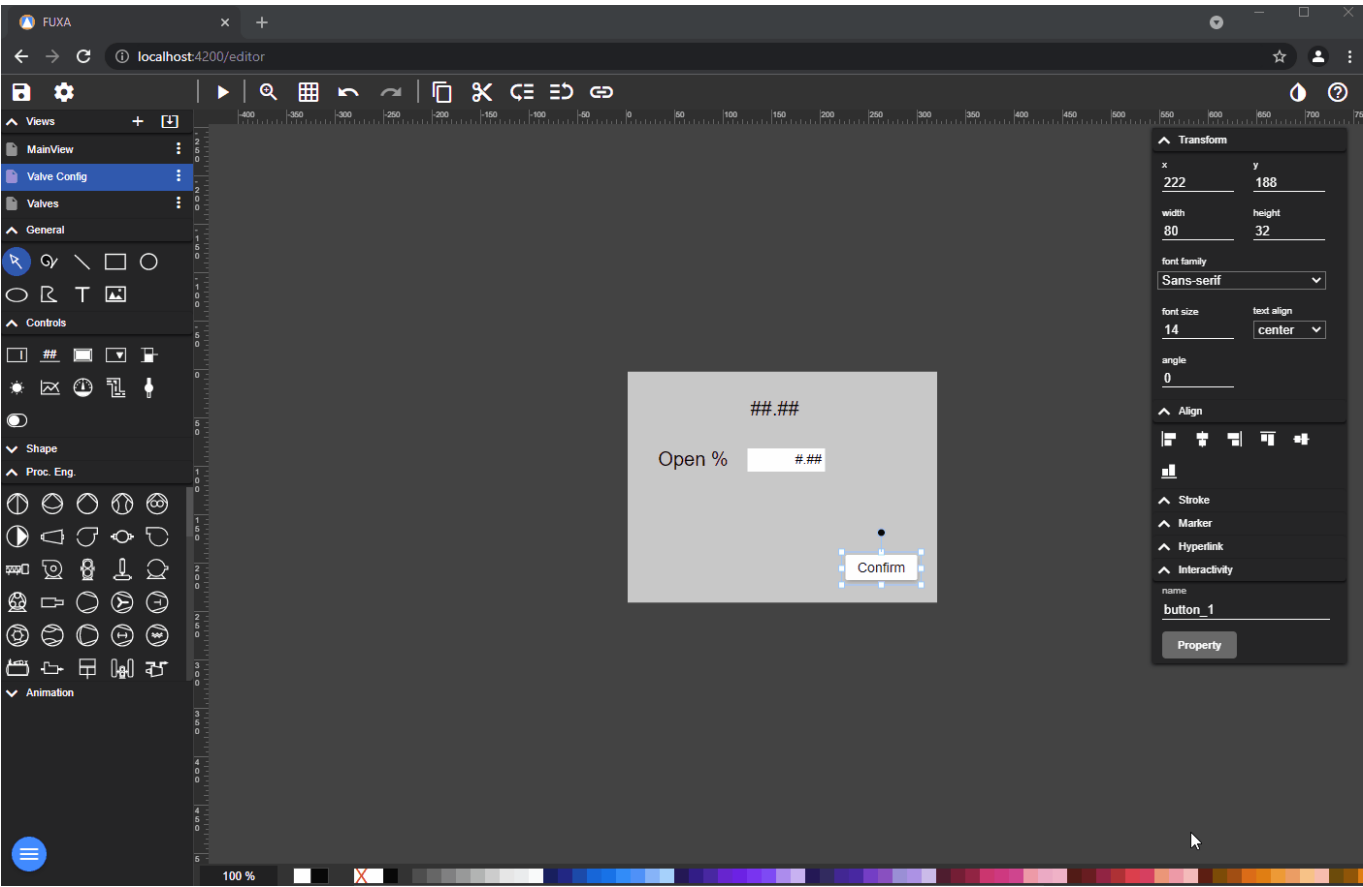
您需要将设备定义为**内部**设备，并定义一些变量（标签）并将它们绑定到可复用**视图**的控件。例如，对话框标题和输入值的变量。



然后在每个组件的 **事件**中定义打开对话框的位置，在其中定义内部标签与设备标签的连接。

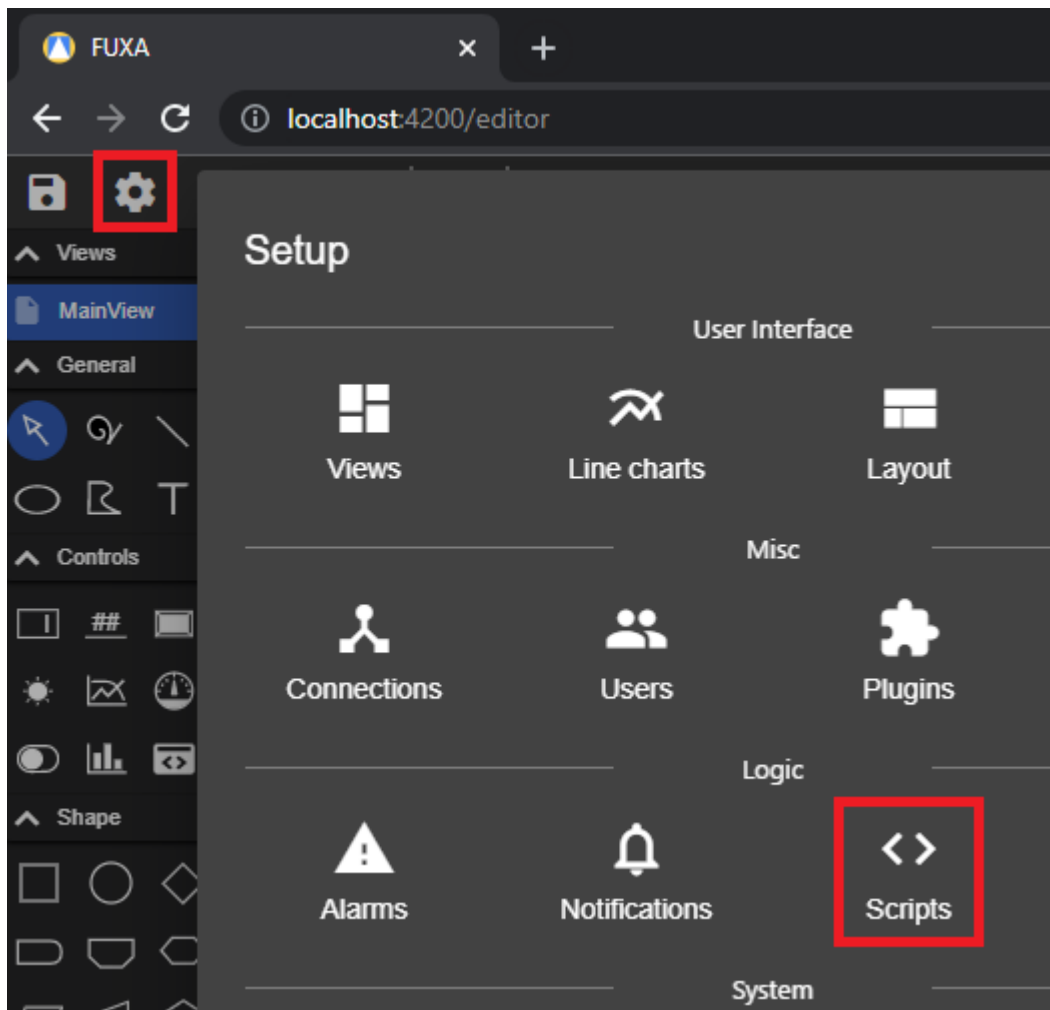


如果需要，您还可以添加确认按钮。



18. 脚本配置

要配置脚本，请在编辑器中进入 **Scripts**。

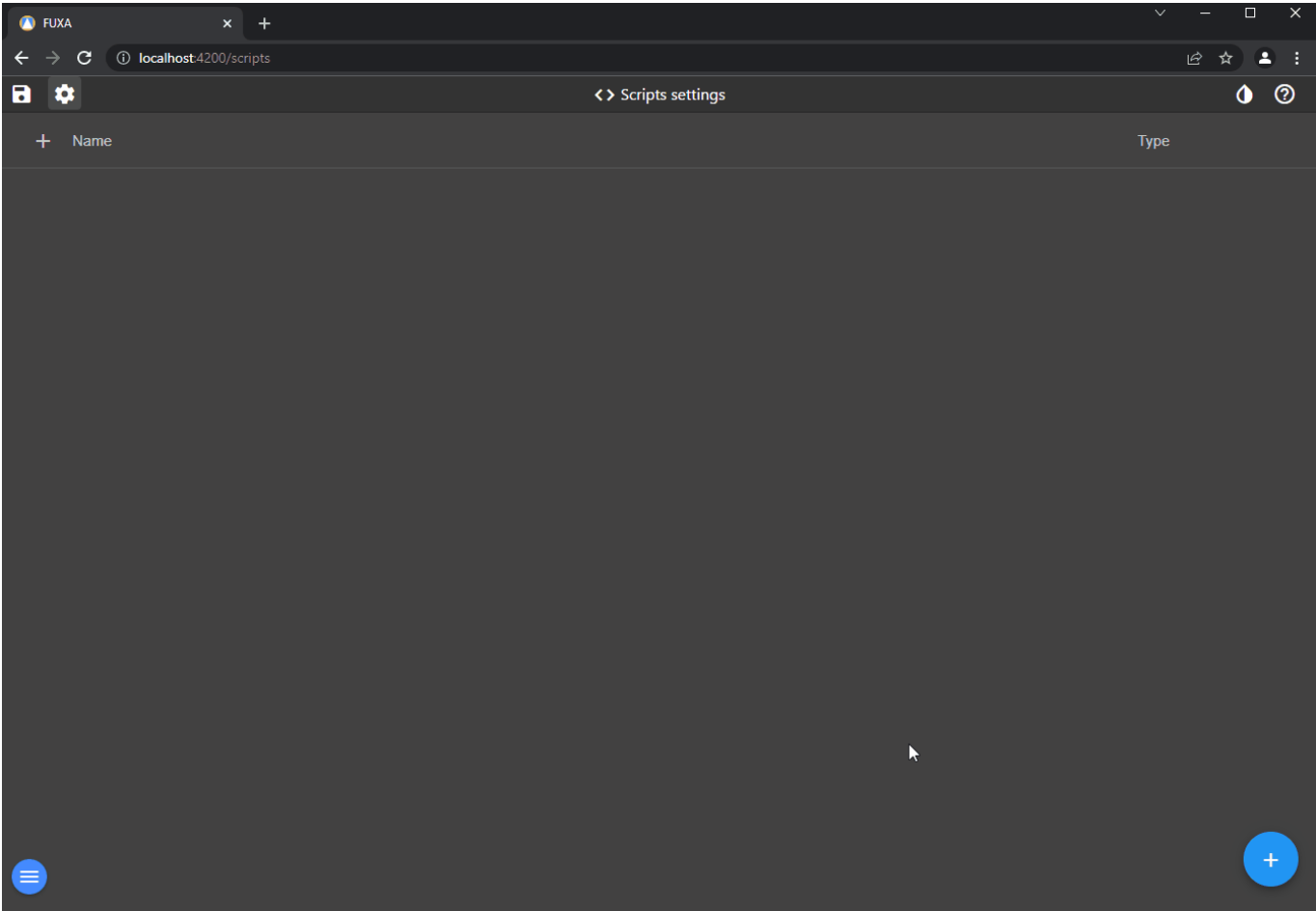


添加新脚本（脚本是一个 javascript 函数），配置函数名和参数，有两种类型的参数：

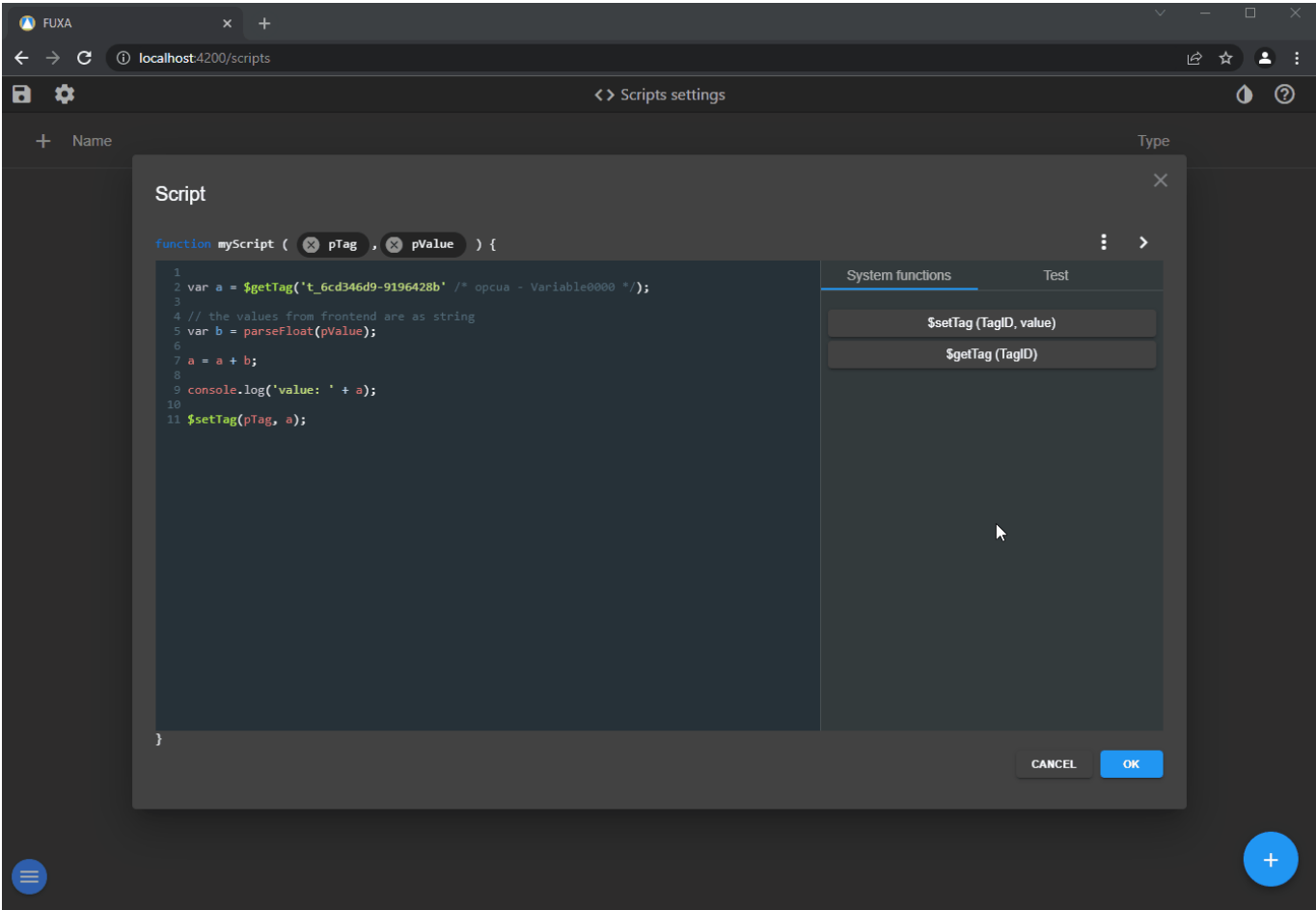
- 标签 ID
- 值，数字或字符串

然后在函数中编写您的逻辑。您可以使用的系统调用：

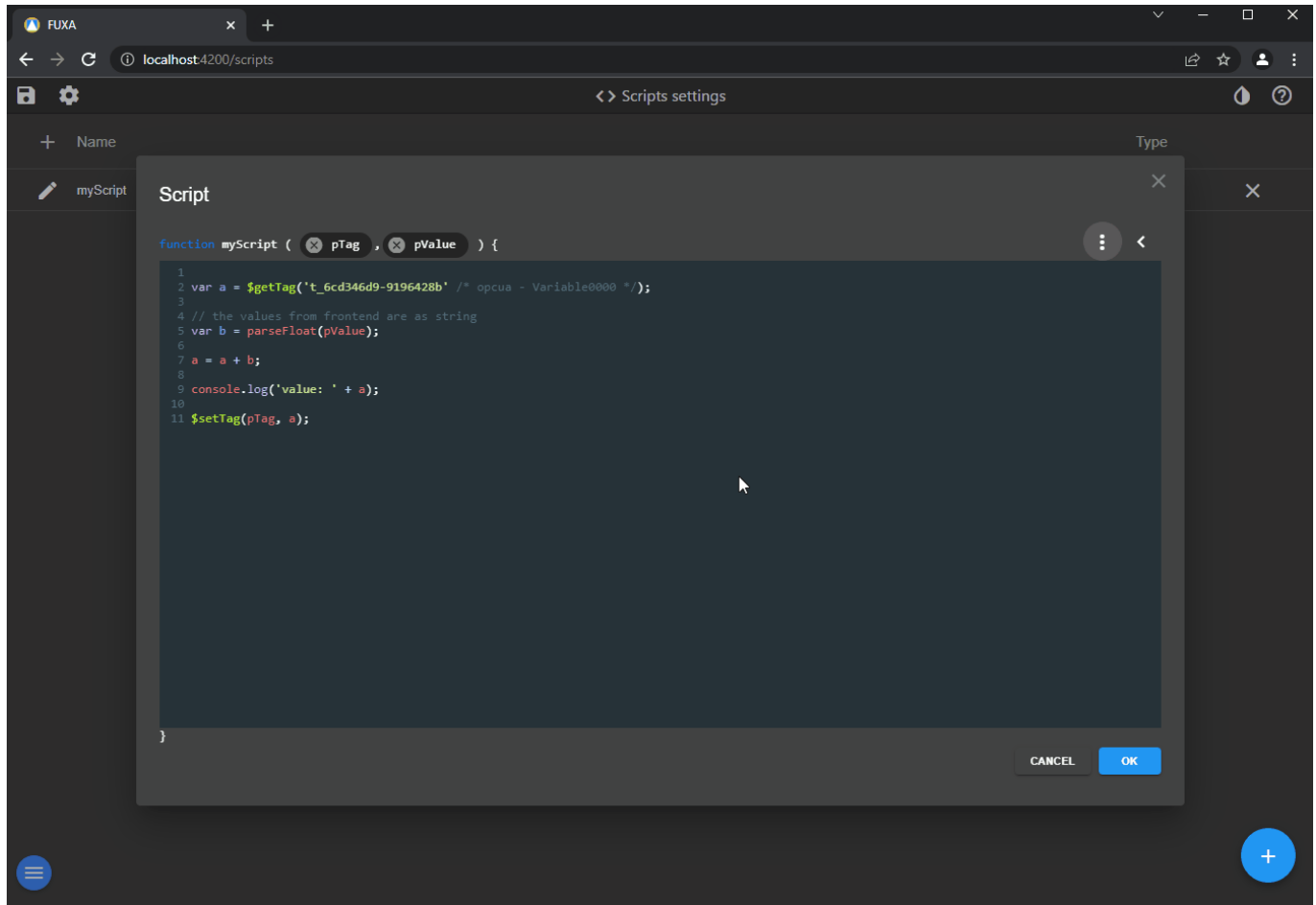
- `$setTag`，设置标签的值
- `$getTag`，获取当前标签的值



您可以测试脚本，使用 console.log 方法验证。



从 GUI 中，您可以在 **Events** 中配置脚本调用。



如果您在脚本中使用间隔，正确处理它们很重要。例如，如果您在脚本中设置 `setInterval` 并多次运行测试，这将为每次调用创建一个新间隔，您必须重启 Fuxa 才能清除它。一种处理方法是给间隔分配一个 ID 并进行简单检查。

```
if (typeof globalThis.myTimer === 'undefined') globalThis.myTimer = null;

if (!globalThis.myTimer) globalThis.myTimer = setInterval(myTimerFunction, 1000);

async function myTimerFunction() {
  //myTimer code here every 1 sec
}
```

19. ODBC 配置

建议使用 Docker 版本，因为 ODBC 驱动程序已预装并准备就绪。

安装 ODBC 驱动程序

可以使用的可用驱动程序及其定义名称可在此处找到：[ODBC Driver ini](#)

如果通过 NPM 在基于 Debian 的 Linux 系统上安装，您可以尝试按照以下步骤手动为系统安装 ODBC 驱动程序：

- 1: `sudo apt-get update && sudo apt-get install -y unixodbc unixodbc-dev`
- 2: 将这两个文件从[这里](#)复制到您的系统
- 3: 进入下载文件的目录，使脚本可执行并安装 ini 文件

```
sudo chmod +x install_odbc_drivers.sh
sudo ./install_odbc_drivers.sh
sudo cp odbcinst.ini /etc/odbcinst.ini
```

- 4: 测试已安装的驱动程序，您可以使用 unixODBC 连接，修改连接字符串以适合您的数据库

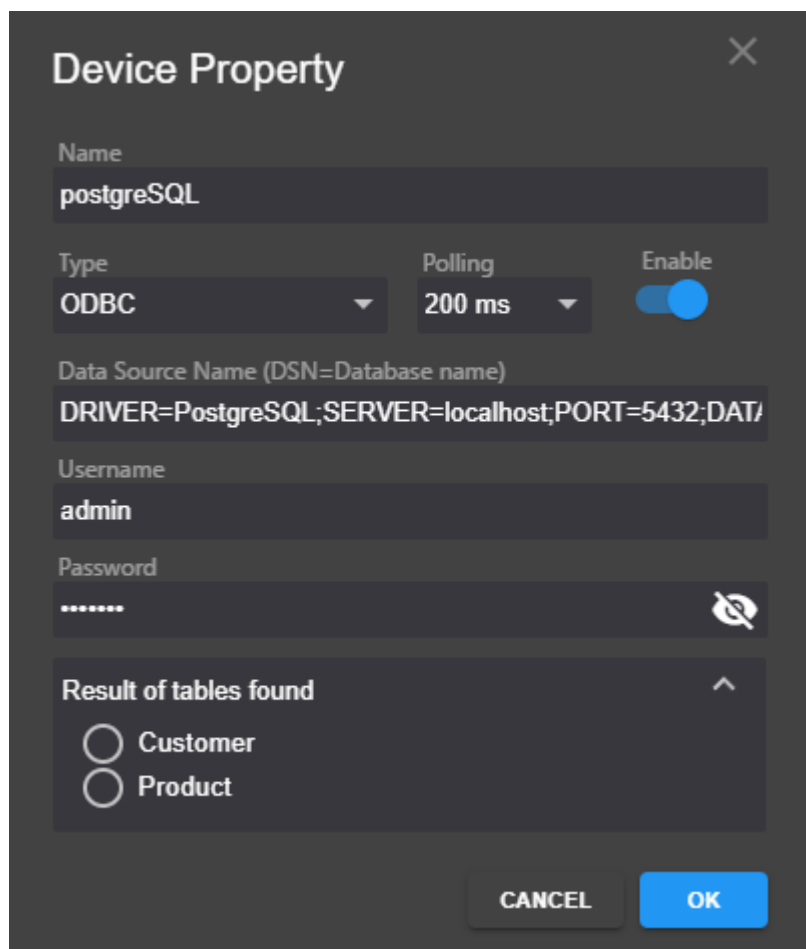
```
sudo myodbc-installer -s -a -c2 -n "test" -t
"DRIVER=MySQL;SERVER=YourIP;PORT=3306;DATABASE=testDB;UID=User;PWD=MyPass"

sudo isql test
```

如何使用 ODBC

首先，您需要通过在 Fuxa 中添加设备并选择 ODBC 来创建与数据库的连接。

如果您使用定义的 DSN，可以输入该名称，或者只需在如图所示的同一字段中添加完整连接字符串。



The screenshot shows the 'Device Property' dialog box with the following fields and options:

- Name:** postgresSQL
- Type:** ODBC (selected from a dropdown)
- Polling:** 200 ms (selected from a dropdown)
- Enable:** A blue toggle switch is turned on.
- Data Source Name (DSN=Database name):** DRIVER=PostgreSQL;SERVER=localhost;PORT=5432;DAT/
- Username:** admin
- Password:** A masked field with dots and an eye icon to toggle visibility.
- Result of tables found:** A list with two items: 'Customer' and 'Product', each with a radio button.
- Buttons:** CANCEL and OK at the bottom right.

以下是一些不同数据库的连接字符串：

```
DRIVER=PostgreSQL;SERVER=Your_DB_IP;PORT=5432;DATABASE=testDB
```

```
DRIVER=MySQL;SERVER=Your_DB_IP;PORT=3306;DATABASE=testDB;SSLMODE=DISABLED
```

MySQL 由于 SSLMODE 可能会导致连接问题，请尝试在连接字符串中不带 SSLMODE 并尝试启用和禁用。

```
SSLMODE=ENABLE SSLMODE=DISABLED
```

将 testDB 替换为您的实际数据库名称

创建一个服务器端脚本进行测试，您可以使用那里的测试选项卡和控制台显示结果。

首先，您需要通过设备获取 ODBC 连接，在示例中它被命名为 postgresQL，如上图所示。

```
// Initialize the device, sane name as connection
let myDevice = await $getDevice('postgresQL', true);
```

从数据库读取数据

```
let result = await myDevice.pool.query(`SELECT * FROM "testTable"`);
console.log(JSON.stringify(result));
```

一些重要说明：

根据您的数据库，您可能需要删除引号 "testTable" 改为 testTable 或添加模式 "DB_Schema"."testTable"

如果您需要在另一个脚本中使用 ODBC，您不能使用相同的连接名称：

脚本 1 `let myDevice1 = await $getDevice('postgresQL', true);`

脚本 2 `let myDevice2 = await $getDevice('postgresQL', true);`

完整示例

此示例还每 100ms 轮询标签并使用触发器执行 SQL 查询，请注意不要使用此方法使系统过载，理想解决方案是创建事件监听器用于标签，我们可以使用简单的 addEventListener 方法而不需要轮询。

还有一个 1 秒循环从数据库更新值并将它们推送到 Fuxa UI 中的表格

有一个名为 query manager 的函数，它处理触发器和一次性执行

注意：您可能需要重启 Fuxa 才能使其正常工作，每次修改脚本时有时它没问题，有时需要重启，所以如果您有任何奇怪的问题，请在创建问题之前尝试重启

创建一个服务器端脚本并将其设置为启动时运行

```
// Query Manager, function to provide one shot based on trigger event
async function createQueryManager(device) {
  let lastTriggerState = false;
  return async function(trigger, sqlQuery) {
    if (trigger && trigger !== lastTriggerState) {
```

```

        try {
            const result = await device.pool.query(sqlQuery);
            lastTriggerState = trigger;
            return result;
        } catch (error) {
            return 'Error executing query';
        }
    }
    lastTriggerState = trigger;
};
}

// Initialize the device, sane name as connection
let myDevice = await $getDevice('postgreSQL', true);

// Create query Manager for each query type, retains query data
let executeInsertQuery = await createQueryManager(myDevice); // Instance Insert
let executeSelectQuery = await createQueryManager(myDevice); // Instance Select

// Global Variables to retain Data
let selResult;

// 100ms loop catch Tag Events
let myLoop100ms = setInterval(loop100ms, 100);

async function loop100ms() {

    let customerName      = $getTag($getTagId('customerName'));
    let customerPhone      = $getTag($getTagId('customerPhone'));
    let customerEmail      = $getTag($getTagId('customerEmail'));
    let customerAge        = $getTag($getTagId('customerAge'));
    let execSaveCustomer    = $getTag($getTagId('execSaveCustomer'));

    // Call Query Manager Instance Function Every 100ms, first parameter is the
    // trigger to execute the query once with a one shot
    await executeInsertQuery(
        execSaveCustomer,
        `INSERT INTO "testData"."Customer" ("Name", "Phone", "Email", "Age")
VALUES ('${customerName}', '${customerPhone}', '${customerEmail}',
${customerAge})`
    );

    // Testing second instance calling same function type
    //selResult = await executeSelectQuery(
    //    execSaveCustomer,
    //    'SELECT * FROM "testData"."Customer"'
    //);
    //
    //$$setTag($getTagId('customerDataArray'), JSON.stringify(selResult));
}

// 1 sec loop update data from DB
let myLoop1sec = setInterval(loop1sec, 1000);

```

```

async function loop1sec() {
  // No Need to use query manager here as we are reading from DB every 1 sec to
  update Data for the Table
  selResult = await myDevice.pool.query('SELECT * FROM "testData"."Customer"');
  $setTag($getTagId('customerDataArray'), JSON.stringify(selResult));
}

```

创建一个客户端侧脚本，间隔为 1 秒，此脚本将把数据放入表格

```

let customerData = JSON.parse($getTag($getTagId('customerDataArray' )));

// Column ID's must match DB column
var tableData = {
  columns: [{
    id: 'Name',
    label: 'Name'
  }, {
    id: 'Phone',
    label: 'Phone'
  }, {
    id: 'Email',
    label: 'Email'
  }, {
    id: 'Age',
    label: 'Age'
  }],
  rows: customerData
};

// Name of table used in Fuxa
$invokeObject('customerTable', 'setTableAndData', tableData);

```

20. 小部件

Fuxa 小部件使用纯 SVG 和 JavaScript (通过 script 标签)

```

<svg>
  SVG Content Here
  <script>
    JS Script Content Here
  </script>
</svg>

```

我们可以通过使用一些简单函数并声明变量为全局变量来在 SVG 脚本之间传输数据

```
_pb_ = bool    参数 ( true 或 false )
_pn_ = number 参数 ( Int, Float, Real 等 )
_ps_ = string 参数 ( 字符串可以直接输入 )
_pc_ = color 参数, 十六进制颜色代码 ( #00ff00ff )
```

同样重要的是，变量必须包含在注释 `///export-start 和 ///export-end 中`

```
///export-start
let _pn_value = 50;
///export-end
```

这些变量现在将在 Fuxa 的 SVG 属性面板中可用，您可以在其中绑定 Fuxa 标签

为了在小部件/SVG 之间传输数据，我们需要使用 2 个函数：

从 Fuxa 向 SVG 发送值

```
function postValue(id, value) {
  console.error('Not defined!');
}
```

您需要调用该函数，ID 需要与确切的变量名匹配

```
postValue('_pn_value', someNewValue);
```

从 Fuxa 向 SVG 接收值

```
function putValue(id, value) {
  if (id === '_pn_value') {
    callFunction(value);
    newVar = value;
  }
}
```

这里我们等待 Fuxa 中调用的函数并检查我们想要的 ID，这是定义的精确变量名

如果您使用 `setInterval`，当您更改页面或进入编辑器时，您还需要清除间隔，如果不这样做，小部件将因为另一个 `setInterval` 被调用而损坏。在我们的示例中，我们检查对象是否存在并使用 `MutationObserver`

```
const checkDestroy = document.getElementById('svgIdName'); // Important must be
name of the SVG!
if (!checkDestroy) {
```

```
clearBlinking();
return;
}
```

以及 MutationObserver

```
// Set up the MutationObserver to watch for removal of the SVG element
const observer = new MutationObserver((mutations) => {
  mutations.forEach((mutation) => {
    mutation.removedNodes.forEach((node) => {
      if (node.id === 'svgIdName') { // Important must be name of the SVG!
        clearBlinking();
        observer.disconnect();
      }
    });
  });
});
// Start observing the body or a parent element of the SVG
observer.observe(document.body, { childList: true, subtree: true });
```

重要的是您在 SVG 文件顶部使用 SVG Id `<svg id=svgIdName>`

您还可以在 SVG 中使用 CSS 和完整 JS，并使用标准的 `getElementById` 和 `addEventListener` 访问元素/ID

有关完整的工作示例，请参阅小部件部分中的示例

https://github.com/frangoteam/FUXA/tree/master/server/_widgets

21. WebSocket

WebSocket 是与其他 Web 应用程序（如 Node-Red）通信的简单好方法。我们可以直接在 Fuxa 脚本中使用 WebSocket。

这是一个双向数据传输的示例，我们将所有 Fuxa 标签和值包装到带有时间戳和有效负载的 JSON 数据对象中。我们使用 for 循环获取所有标签值并插入有效负载，我们也使用 for 循环写入从 WebSocket 接收的标签值。

脚本必须是服务器端脚本并设置为启动时运行。您必须已经有一个正在运行的 WebSocket 服务器，因为下面的代码是 WebSocket 客户端代码。

如果需要，也可以在 Fuxa 脚本中创建 WebSocket 服务器。下面的代码仅涵盖客户端连接到服务器。

```
const WebSocket = require('/usr/src/app/FUXA/server/node_modules/ws');
let ws; // Declare ws Web Socket globally to manage the connection

const WebSocketUrl = 'ws://127.0.0.1:1880'; // Url to WebSocket Server

// List of PLC/FUXA Tags
const tagNames = [
  'yourTagName1',
```



```
    'yourTagName2',
    'yourTagName3',
    'yourTagName4',
    'yourTagName5'
  ];

  // Function to get tag values and create the JSON payload
  async function createPayload() {
    let payload = {};

    for (const tagName of tagNames) {
      let tag = await $getTag($getTagId(tagName));
      payload[tagName] = tag;
    }

    return {
      data: {
        timestamp: new Date().toISOString(),
        payload: payload
      }
    };
  }

  // Function to send data to WebSocket server
  function sendData() {
    createPayload().then((payload) => {
      if (ws && ws.readyState === WebSocket.OPEN) {
        ws.send(JSON.stringify(payload));
      } else {
        //console.log('WebSocket is not open. Skipping sending data.');
      }
    });
  }

  // Open WebSocket connection
  function openWebSocketConnection() {
    ws = new WebSocket(WebSocketUrl);

    ws.on('open', () => {
      console.log('WebSocket connection established');
    });

    ws.on('message', (message) => {
      try {
        let receivedData = JSON.parse(message);
        if (receivedData.data && receivedData.data.payload) {
          for (const tagName in receivedData.data.payload) {
            if (tagNames.includes(tagName)) {
              $setTag($getTagId(tagName),
receivedData.data.payload[tagName]);
            }
          }
        }
      } catch (error) {
```

```
        console.error('Error parsing message from WebSocket server:', error);
    }
});

ws.on('error', (error) => {
    console.error('WebSocket error:', error);
});

ws.on('close', () => {
    console.log('WebSocket connection closed');
    // Attempt to reconnect after a delay if the connection is closed
    setTimeout(openWebSocketConnection, 5000);
});

}

// Start the WebSocket connection
openWebSocketConnection();

// Set interval to send data every 500ms
if (typeof globalThis.myTimer === 'undefined') globalThis.myTimer = null;

if (!globalThis.myTimer) globalThis.myTimer = setInterval(myTimerFunction, 500);

async function myTimerFunction() {
    sendData();
}
```

22. 设置

服务器设置位于 `server\_appdata\settings.js` 文件中。更改后需要重启服务器。

身份验证

要启用和配置身份验证：

```
secureEnabled: true,           // 启用或禁用
secretCode: 'frangoteam751',   // 用于编码令牌的密钥
tokenExpiresIn: '1h'           // 令牌过期延迟 '1h'=1小时, 60=60秒, '1d'=1天
```

默认用户 'admin' 的密码是 '123456'，您当然可以更改它。

23. 技巧和窍门

编辑器键盘快捷键：

- Ctrl + 左/右：旋转选中的项目
- Ctrl + Shift + 左/右：大步旋转选中的项目
- Shift + O / P：选择上/下一个项目

- Tab / Shift + Tab: 选择上/下一个项目
- Ctrl + 上/下: 中心缩放
- Ctrl + Z / Y: 撤销/重做
- Shift + '用鼠标调整选中的项目大小': 锁定宽度和高度
- Shift + 上/下/左/右: 移动选中的项目
- Shift + 滚轮: 按鼠标位置缩放
- Ctrl + A: 全选
- Ctrl + G: 分组或取消分组选中的项目
- Ctrl + D: 复制选中的项目
- Shift + '绘制线条': 线条水平、垂直、45° 对角渐变
- Ctrl + X: 剪切选中的项目
- Ctrl + C / V: 复制/粘贴选中的项目

powered by **frangoteam**